Lesson 7

Model-Driven Engineering

Prof. Judith Michael | Chair of Programming and Software Engineering

** thanks to Bernhard Rumpe and his team at RWTH Aachen University*



Universität Regensburg

What we will discuss today

- Fundamentals of Model-Driven Engineering
- MDE with the code generator framework MontiGem
- Domain model
- Generated code and product
- GUI modeling & testing
- Example Applications

7.1 Mode-Driven Engineering Fundamentals

Prof. Judith Michael | Chair of Programming and Software Engineering

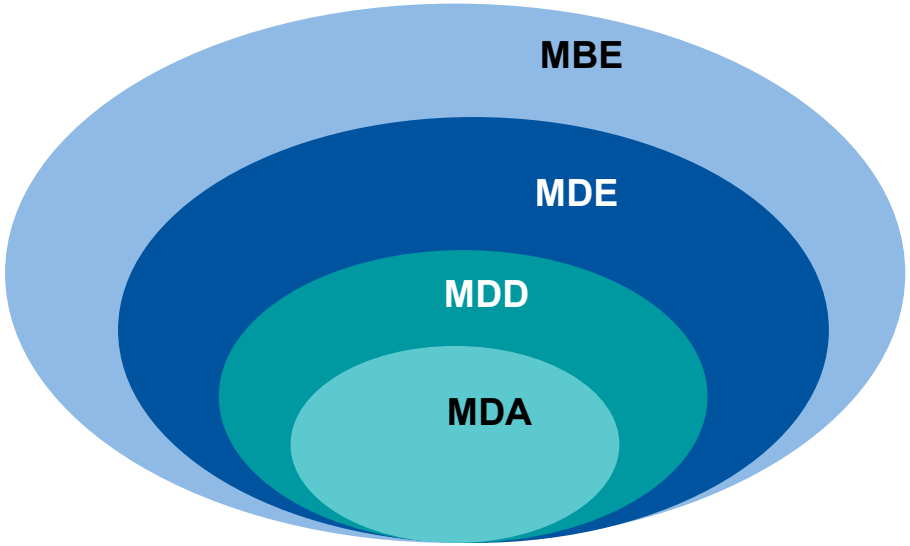


Universität Regensburg

What is Model-Driven Engineering (MDE)?

- A *model-centric* software engineering approach
  - Models are the *basis and driving force* for the engineering process

- Includes
  - Model-driven *requirements engineering*
  - Model-driven *design*
  - Model-driven *software development*
    - Deterministic code generation from models
  - Model-driven *testing*
  - Model-driven *system evolution*
  - Model-driven *reverse engineering*
  - …



- Model-Based Engineering
  - Software models play an important role but are not the key artifact of the engineering process
    - e.g., models specified in design, then used by developers to manually write code (softer than MDE)
- Model-Driven Development
  - Models drive (only) the development process
- Model-Driven Architecture
  - Concept of the OMG – using OMG standards

T. Stahl, M. Völter: Model-Driven Software Development. Wiley, 2006.
J. Cabot: Clarifying concepts: MBE vs MDE vs MDD vs MDA. https://modeling-languages.com/clarifying-concepts-mbe-vs-mde-vs-mdd-vs-mda/
M. Brambilla, J. Cabot, M. Wimmer: Model-Driven Software Engineering in Practice. Springer, 2017

Generative Software Engineering

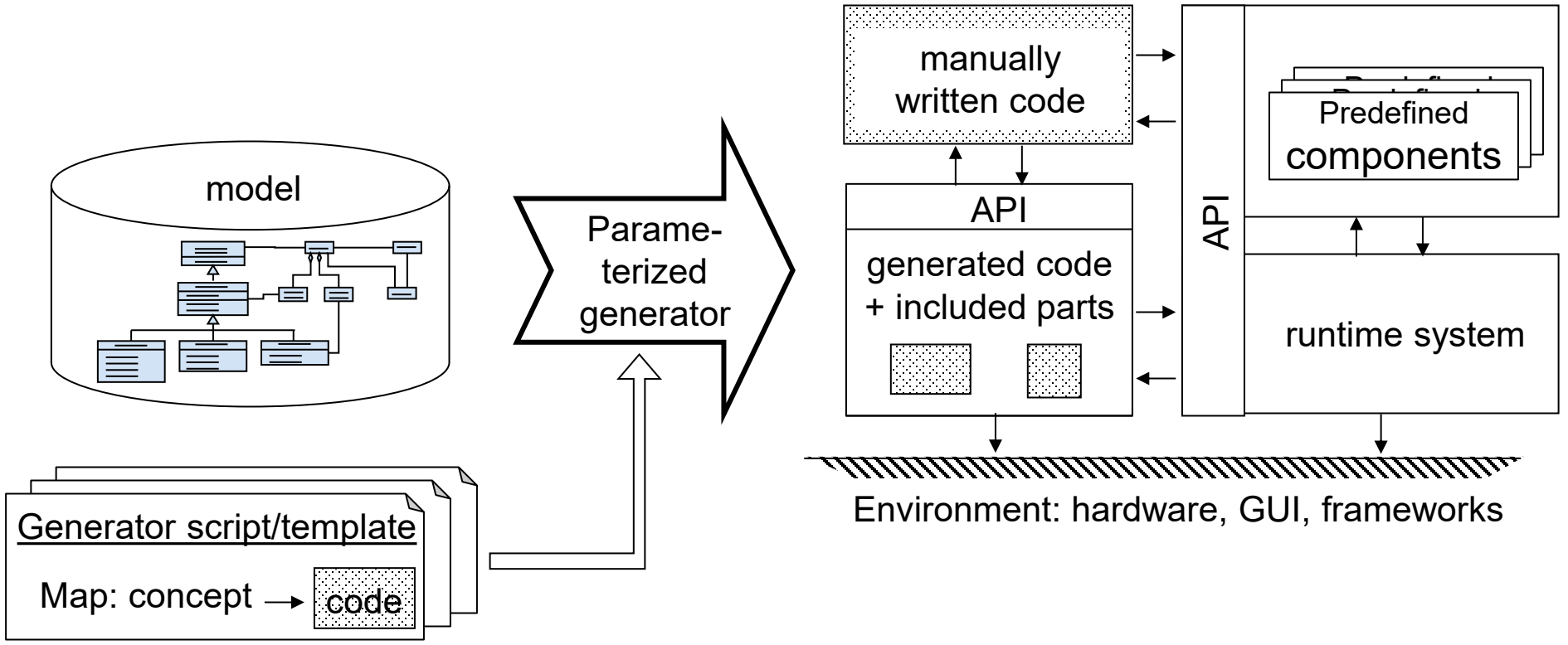
Generative Software Engineering (GSE) is a

- method for efficiently generating (parts of) software using generators
- based on models
  - such as UML or a DSL (domain-specific language)
- with the aim to
  - improving quality and
  - reducing development time.

- Application for:
  - Code generation
  - Generation of test cases (e.g., JUnit test cases)
  - Prototyping
  - Simulation
- Additional reasons:
  - Avoids tedious programming tasks
  - Technology-independent
  - Software stack-independent
  - Allows for various implementation variants



How a Generator Works



7.2 MDE with MontiGem

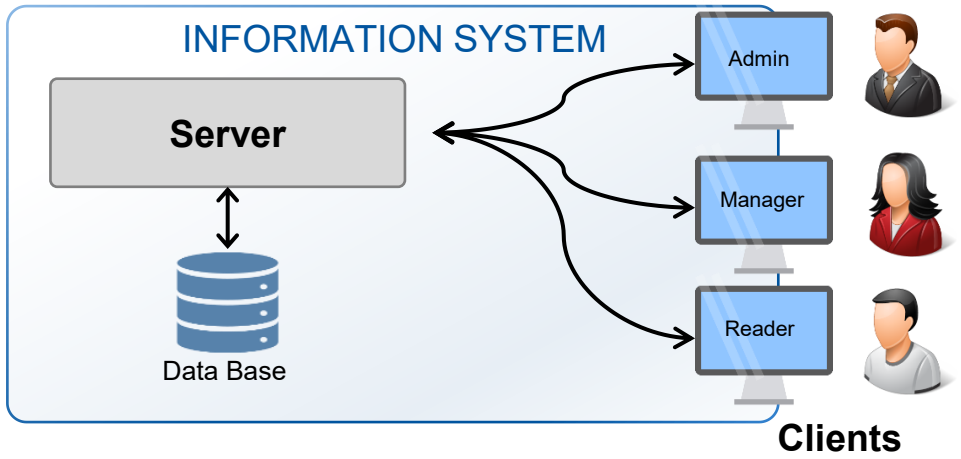
Prof. Judith Michael | Chair of Programming and Software Engineering



Universität Regensburg

Data model-centric applications

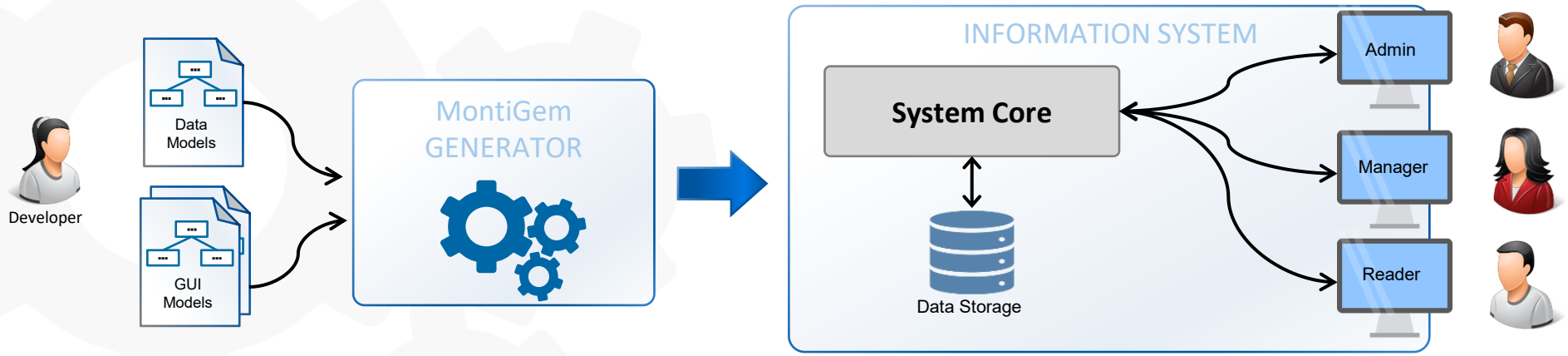
- Information systems
  - is based on information with complex structure (i.e. complex data structure)
  - aims at controlling, accessing, and managing data
  - ensures data consistency, completeness, integrity
  - provides a large set of functionality
  - business process assistance



- Usually:
  - Client-server architecture
  - Lightweight client (part of information system)
  - Elaborated persistence infrastructure

How does MontiGem 3 work?

The **MontiGem** framework provides support to generate large parts of a **web-based** domain-specific **information system** consisting of **data base**, **persistence layer**, **command infrastructure** and **graphical user interfaces** based on the **input** in **models**.



1 Define models
- Required: Class Diagrams (Data structure, subsets for views), GUI Models (Views)
- CD2GUI: only Class Diagrams (Data structure)

2 Generation of the „System Core“
- Database
- Communication Infrastructure between Frontend and Backend
- Graphical User Interfaces

3 Hand-Written Extensions
- In Java, transpiled to TS
- Allow for re-generation
- Customize Appearance
- Add business logic

Data Presentation Functionalities in MontiGem

- Editable *view* per object (all attributes, links)
- List views for *all objects* of a class
- Object *aggregating views*

- *SCRUD* Operations
  - Search, create, read, update, delete

- Multi-user support with integrated user rights administration

- Filter, search
- Undo, redo, …

- APIs for data processing (WebSocket, Rest, …)



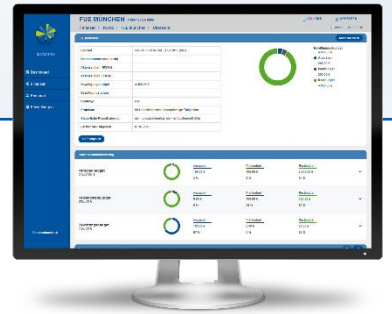
Architecture of the (Generated) Enterprise Information System

Used Technologies and Software Stack

FRONTEND

Caddy HTTP Server
Open-source cross-platform web server software

Angular
TypeScript-based open-source web application framework



Client

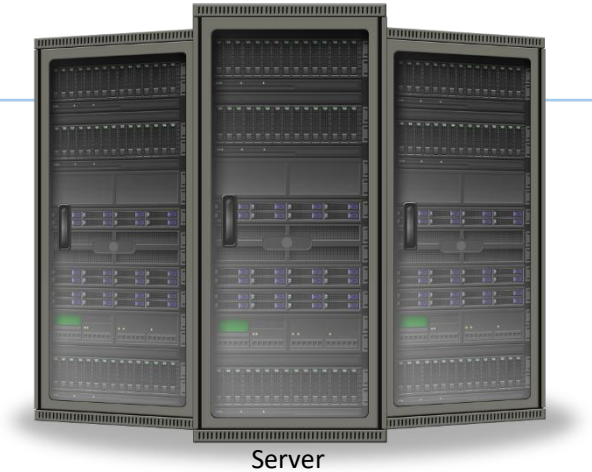
slim frontend:
little logic for the client

BACKEND

Spring Boot
Open-source framework. Serves as application server and manages connections

Spring Security
Future

complete logic for the application



Server

Docker Container

DATABASE

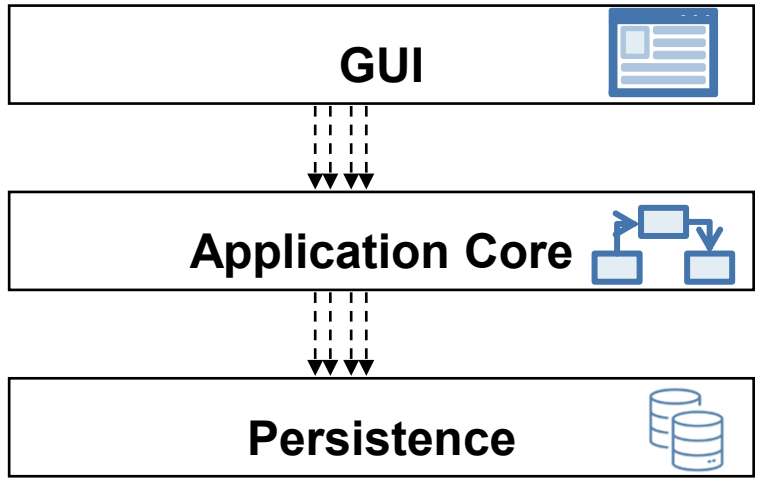
PostgreSQL
Open-source relational database management system

Neo4J
Open-source Graph Database

Architectural Pattern: 3-Tier Architecture

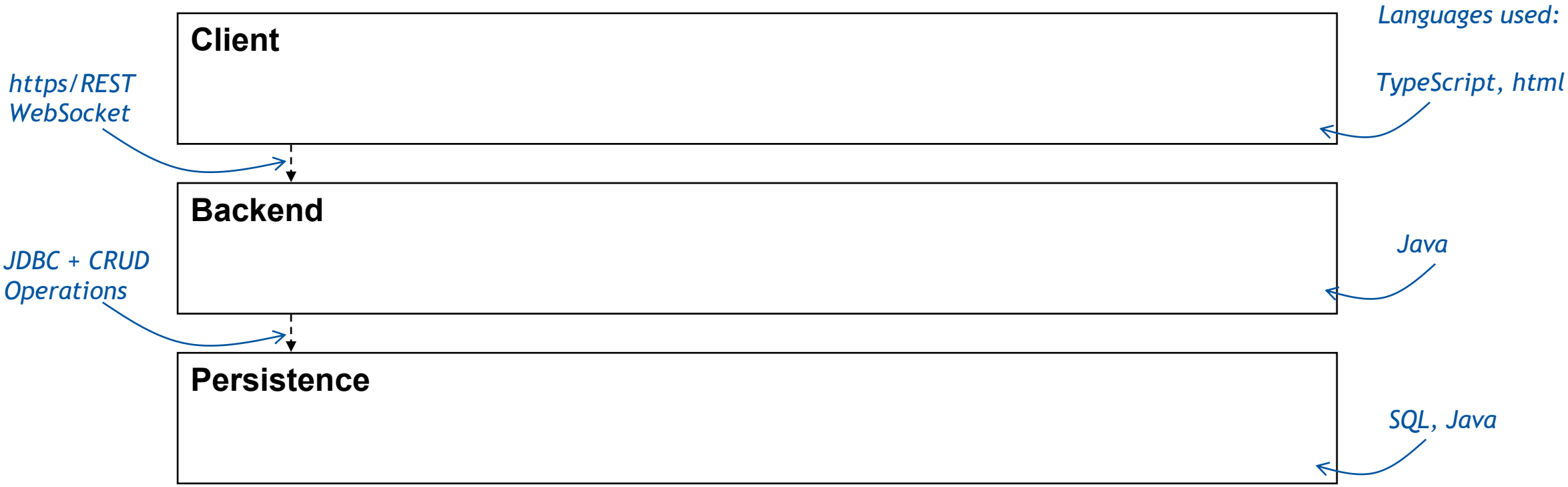
- Products are structured as 3-tier applications
  - **Presentation Tier (GUI):**
    user interface interacts with app core
  - **Application Tier:**
    application core is derived from domain data model
  - **Data Tier:**
    persistence encapsulates access to data storage (includes DB)

Dimension #D1 of product structure:
The architectural pattern / style of the product



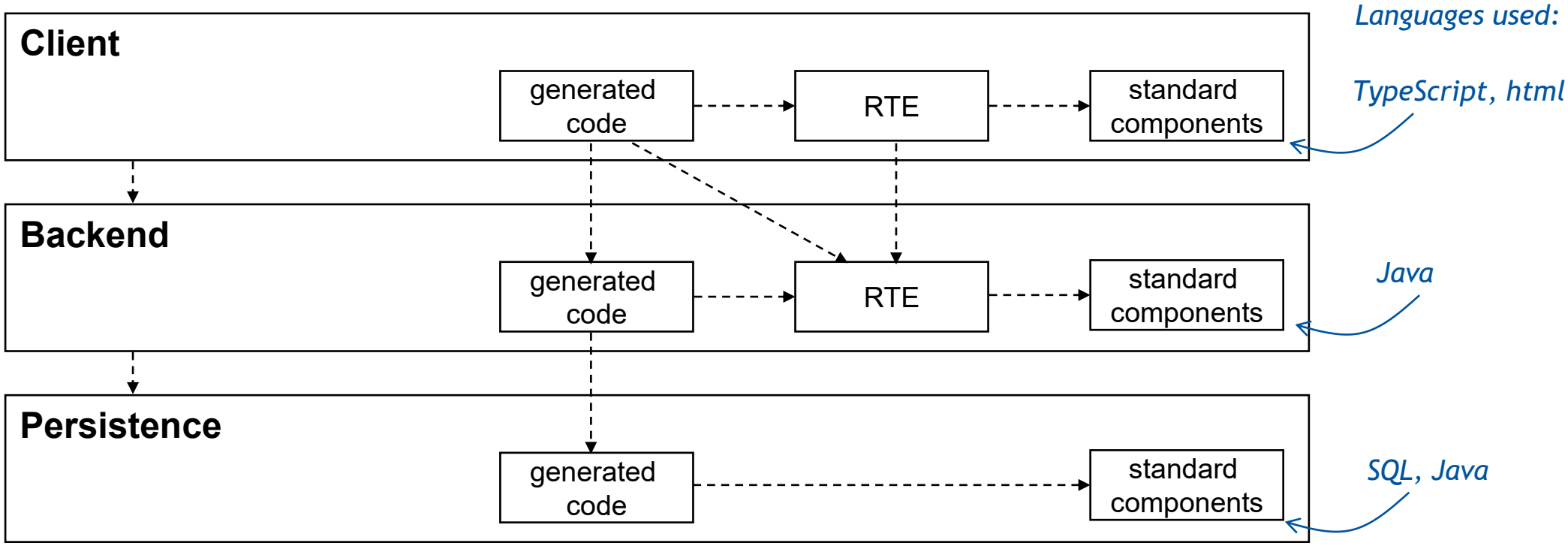
Structure of a Generated Product -1

- MontiGem product architecture has three layers



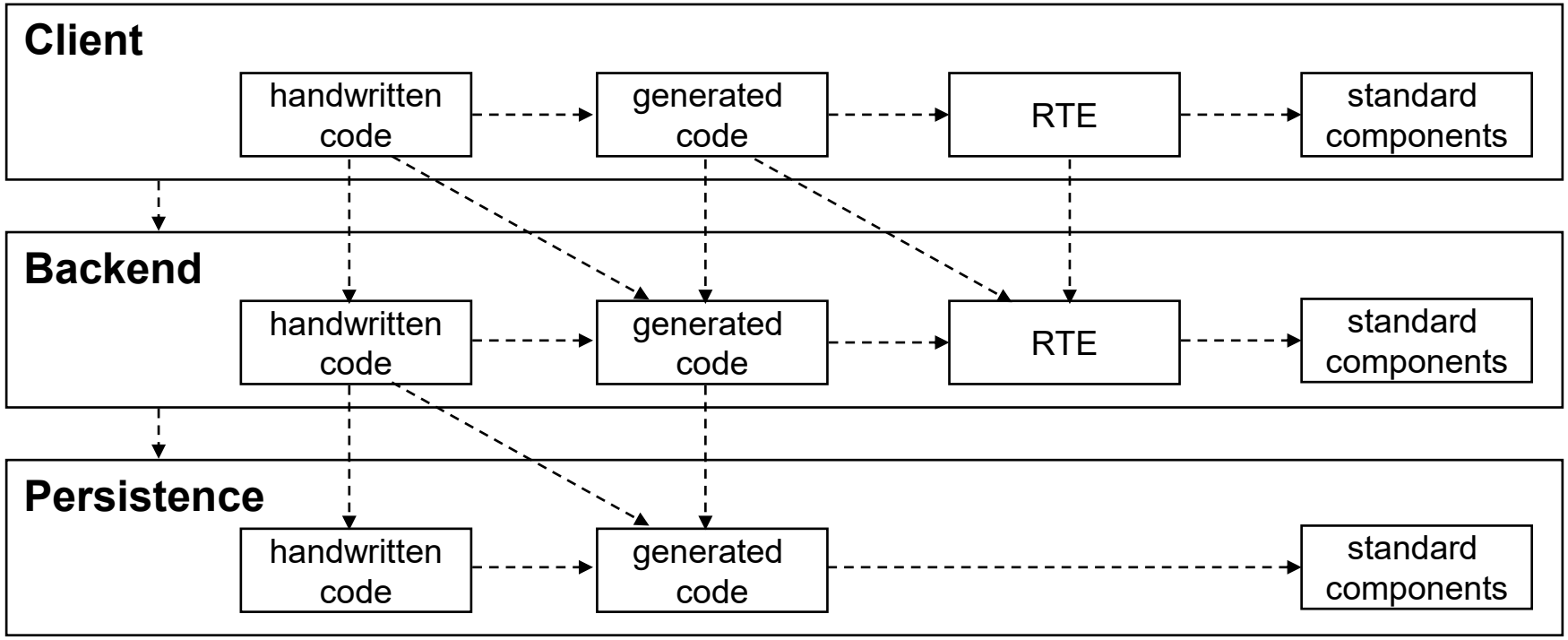
Structure of a Generated Product -2

- Generated code: depends on the models describing the domain (e.g., class diagram)
- RTE: is shipped together with the generator (but independent of the model)



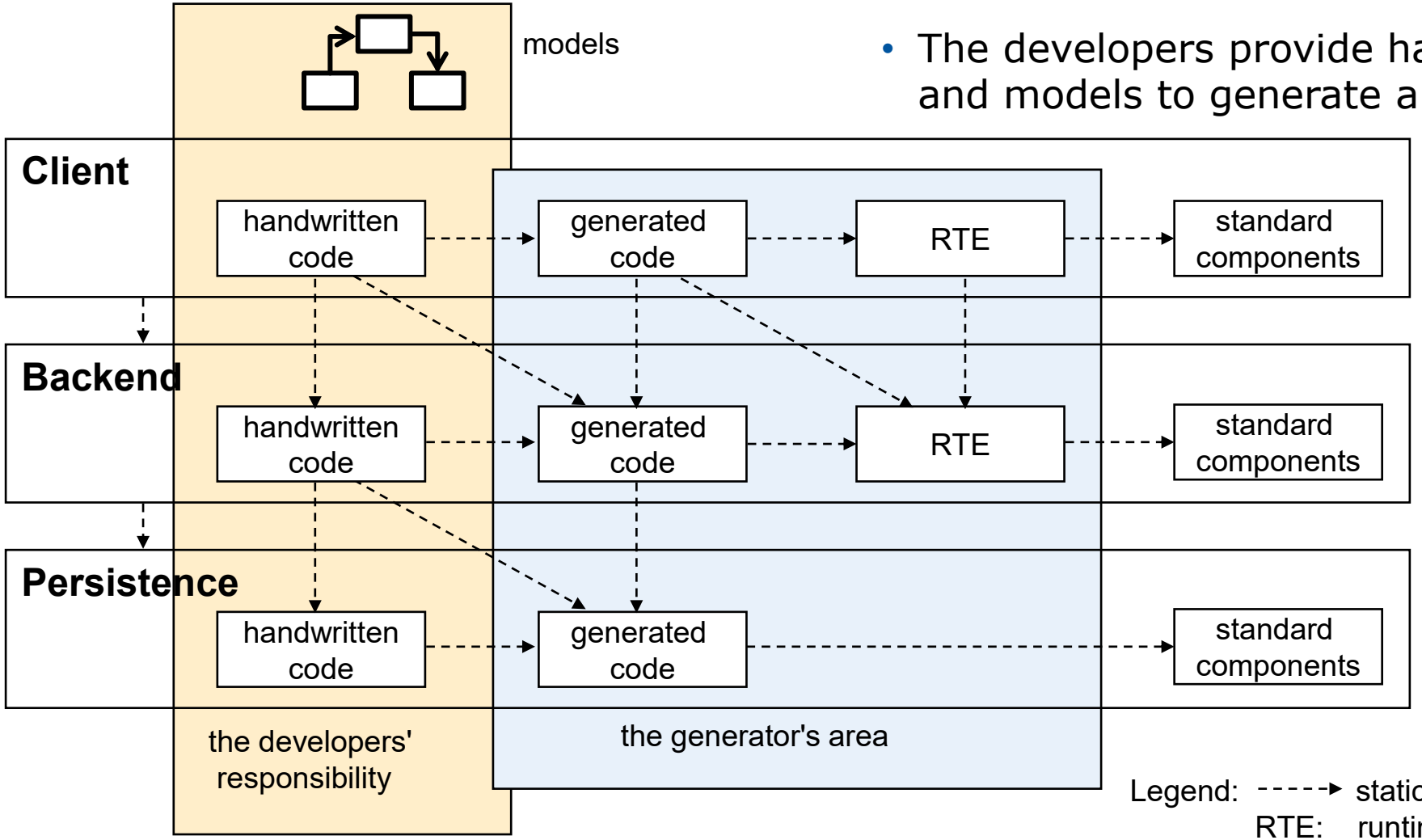
Structure of a Generated Product -3

- The developers also add handwritten code: not everything can/should be generated



Legend: ----→ static knowledge (e.g., import)
RTE: runtime environment

Structure of a Generated Product -4



- The developers provide handwritten code and models to generate a product

Legend: ----→ static knowledge (e.g., import)
RTE: runtime environment

Modeling Languages

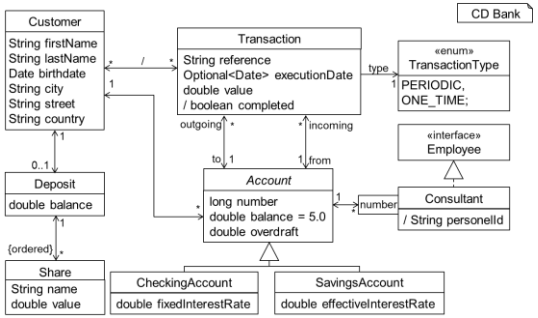
- UML Class Diagram for Analysis (CD4A) is
  - a textual language to describe class diagrams for analysis models

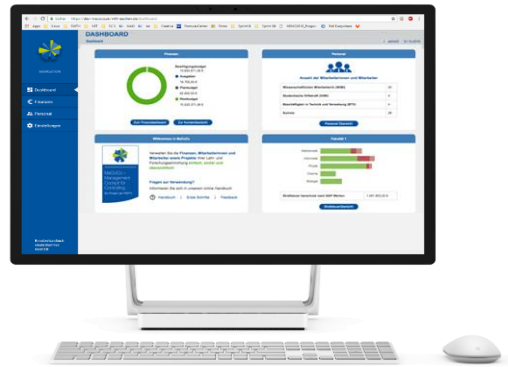
- User Interface Description Language (GUI DSL) is
  - a textual language to describe user interfaces
  - contains data visualizations and interaction components

- UML Object Constraint Language (OCL/P) is
  - constraints for objects and attributes
  - defines validation logic to ensure data integrity

- UML Activity Diagram Language (ADJava) is
  - for process automation, activity tracking

- Handwritten Code in
  - Java for the Server
  - (MontiGem: TypeScript for the Presentation Layer)



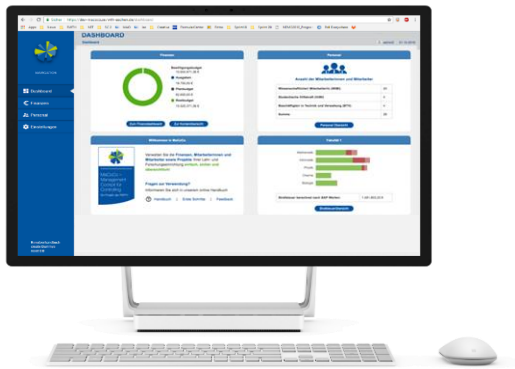


Adaptation of the Code Generation Process

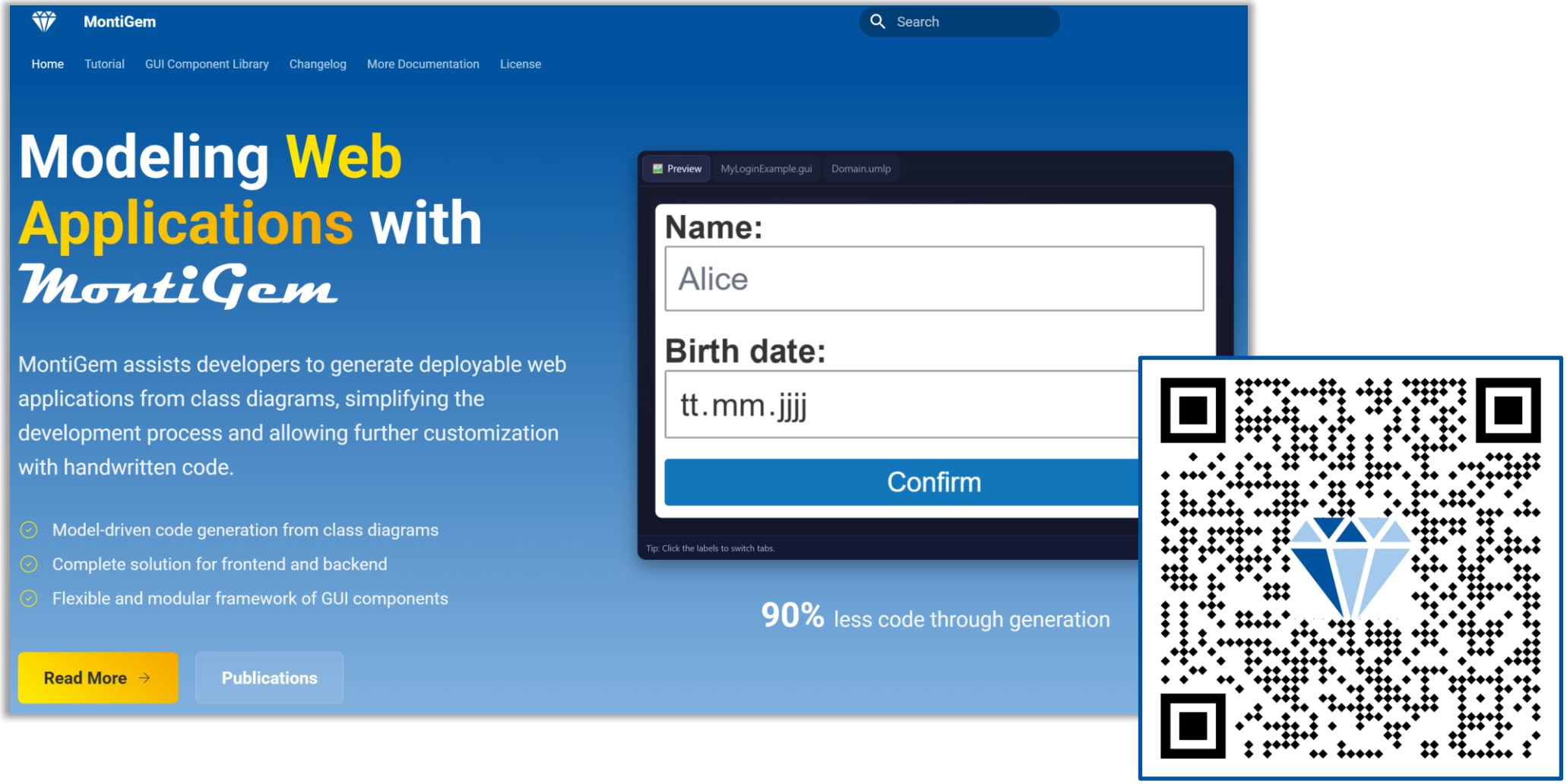
- Adaptation of a code generator is necessary to
  - facilitate tool reuse
  - because customizing generated source code is limited



- Transformation- and Template-based code generation supports adaptation via
  - template extensions (additional templates)
  - hook points (dedicated spots in templates)
  - manually-written transformations



More information on the MontiGem Website



https://monticore.pages.rwth-aachen.de/umlp/index.html

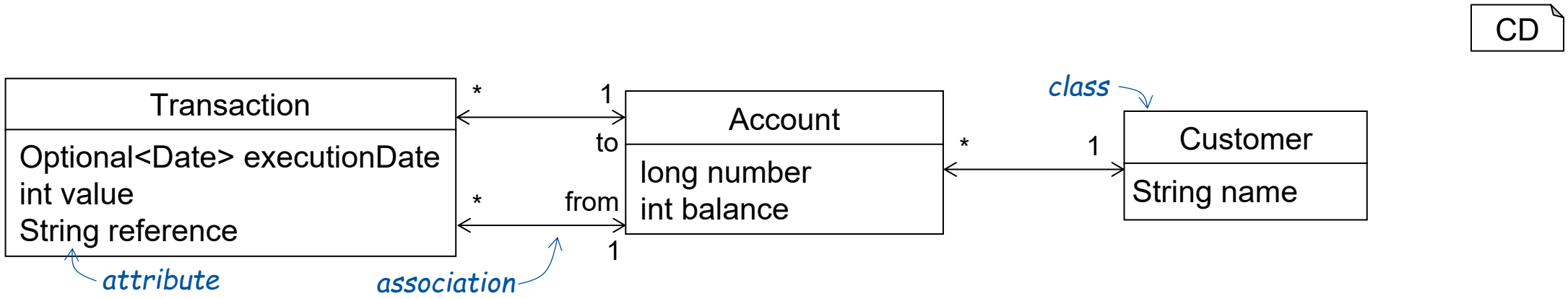
7.3 Domain modeling

Prof. Judith Michael | Chair of Programming and Software Engineering



Universität Regensburg

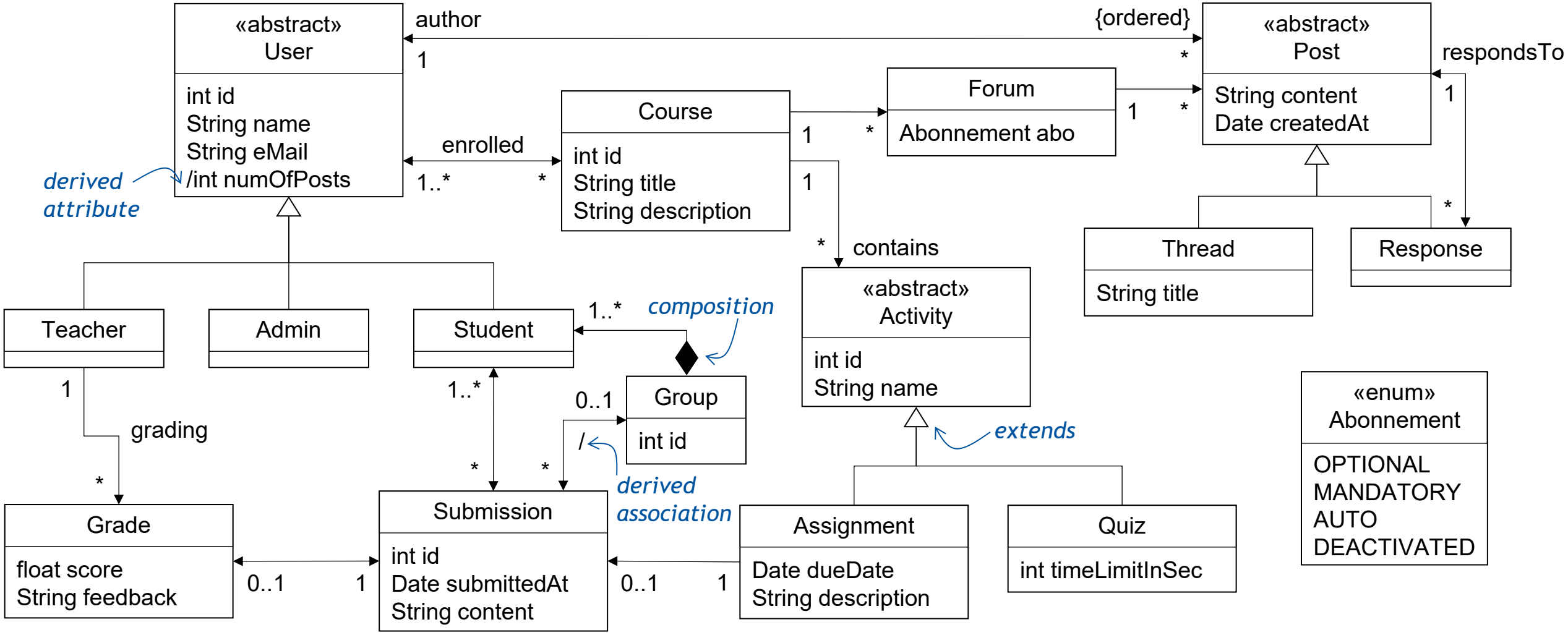
CD4A (Class Diagrams for Analysis)



- Analysis model
  - represents structure of the managed data typical a copy of the real world
  - created in early analysis activities

- UML Class Diagram for Analysis (CD4A)
  = textual modeling language for analysis models that supports
  - classes, interfaces, abstract classes, and enumerations
  - associations, qualified associations, and composition
  - external data types

Example "Moodle"



Concepts we know in CDs for Analysis

- *Classes*
  - Interfaces, abstract classes, enumerations
- *Attributes*
  - Name + Type
  - derived attributes
- *Associations*
  - Name, role names, cardinalities

- *Compositions*
- *Qualified associations*
  - Qualifier
- {*ordered*} associations
- *Stereotypes* like «singleton»

- In Analysis we *do not have*:
  - Methods, Visibility

We Use a Textual Notation for CDs

See another example in Lesson 2

- A class diagram thus becomes a text-file of this form and is stored in **`Moodle.cd`**

CD

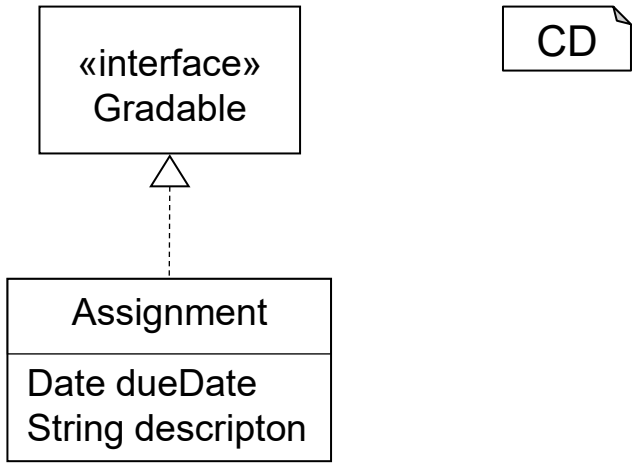
```
1  classdiagram Moodle {
2
3    class {...}
4
5    class {...}
6
7    enum {...}
8
9    association ... ;
10
11   composition ... ;
12 }
```

Classes and Interfaces

See another example in Lesson 2

- Classes and interfaces look similar to Java code
- but neither contain methods nor visibilities.

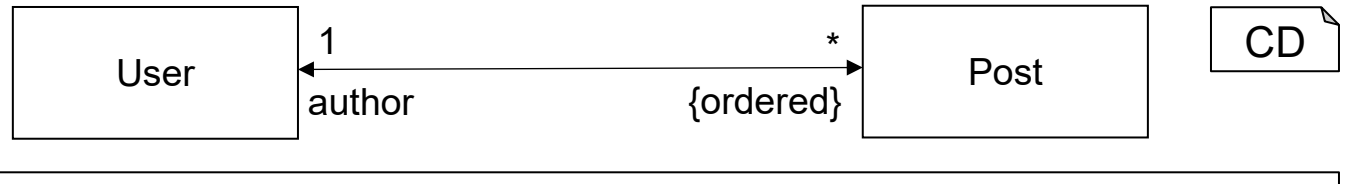
```
1  classdiagram Moodle {
2
3    interface Gradable;
4
5    class Assignment implements Gradable {
6      Date dueDate;
7      String description;
8    }
9  }
```



Associations

See another example in Lesson 2

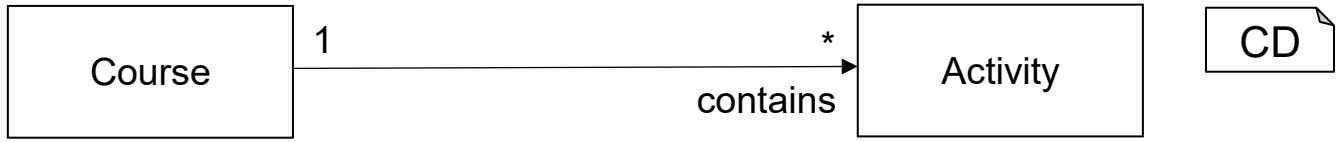
- An association in the CD:
- Navigation directions:
  `->`,
  `<-`,
  `<->`,
  `--`
- Multiplicities:
  `[*]`,
  `[1]`,
  `[0..1]`,
  `[1..*]`
- Optional:
  - Association name
  - Role names
  - Multiplicities (default `[1]`)
  - Ordered keyword (default unordered)




```
association [1] User (author) <-> Post [*] <<ordered>>;
```

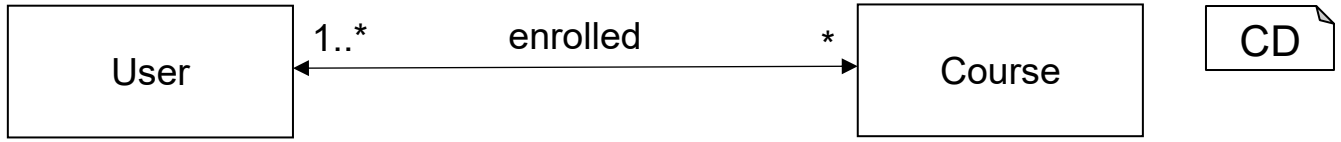
class name

nav. direction



multiplicity

```
association [1] Course -> (contains) Activity [*];
```



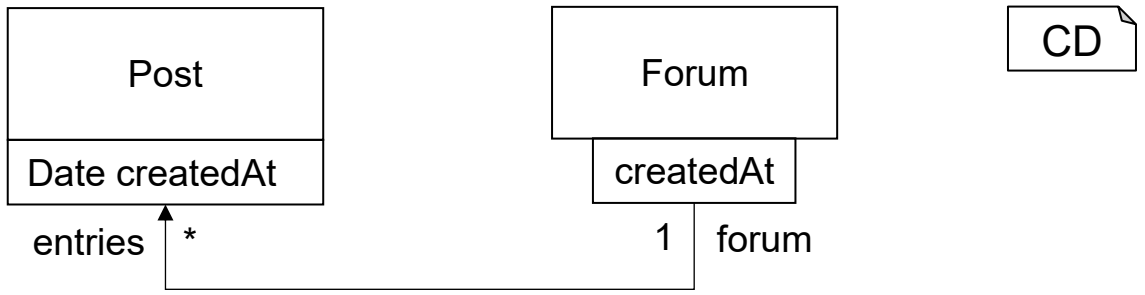
assoc. name

```
association enrolled [1..*] User <-> Course [*];
```

Associations (II)

See another example in Lesson 2

- An association in the CD:




```
association [1] Forum [[createdAt]] (forum) -> (entries) Post [*];
```

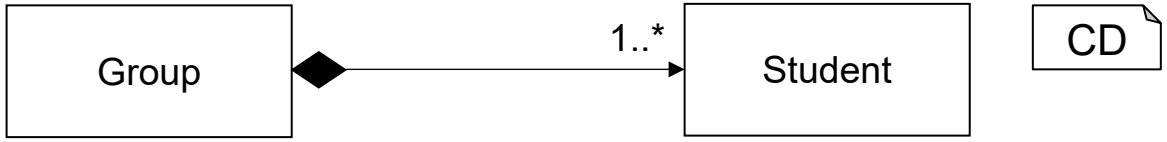
qualifier *role names*

- Role names for navigation
- Qualifier is an
  - (1) a type (notation: `[Type]`) or
  - (2) attribute of the opposite class (notation: `[[attribute]]`)

Composition

See another example in Lesson 2

- A composition in the CD:



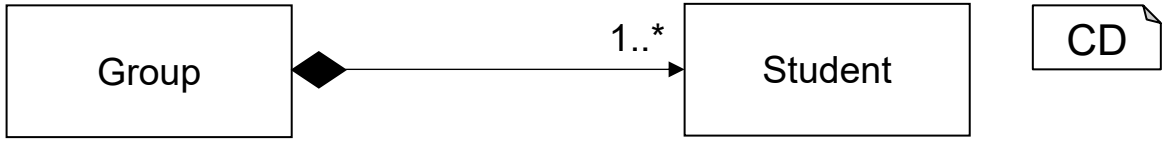

```
composition Group -> Student [1..*];
```

- Component (Student) is part of the composition (Group).
  - A composed object may only occur once in a composition

Composition (II)

See another example in Lesson 2

- Alternative encoding for a composition within the declaration of the composite class (only allows ->):




```
class Group {
  -> Student [1..*];
}
```

- Direct Composition produces a different kind of AST node:
  - requires after-parse transformation of the AST before further processing

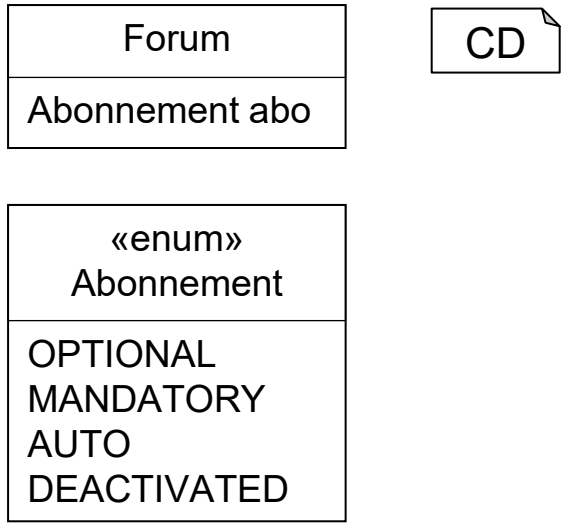
Enumerations

See another example in Lesson 2

- An enumeration (enum) can be used as type:

```
class Forum {
  Abonnement abo;
}


enum Abonnement {
  OPTIONAL, MANDATORY, AUTO, DEACTIVATED;
}
```



Importing Types and Classes

See another example in Lesson 2

- An `import` statement, similar to Java is available
- Import foreign types, e.g. defined as Java classes, like **`Date`** or **`Activity`**
- Imported types are used as class types in the CD
- Note: Library with these types must be known by CD4Analysis tool (e.g. for CoCo check)
- Note: if imported class is used as supertype, some API functions must be provided

```
1  package models;
2  import java.util.Date;
3  import com.se.Moodle.*;
4
5  classdiagram MyExamPreparation {
6
7    class MyExam extends Activity {
8      Date examDate;
9      String room;
10   }
11
12   association grading
13      MyExam -> Grade;
14
15 }
```

Generic Types and Classes

See another example in Lesson 2

- CD4Analysis does not allow to introduce generic types
- It only provides these generic type constructors:
  - `Optional< . >`
  - `List< . >`
  - `Set< . >`
  - `Map< . , . >`

  with one, respectively two type parameters.
- Note: Use of generics with classes is normally not recommended, as associations do have the very(!) same effect. E.g. do not use `Set<Teacher>`, but `association ... -> Teacher [*].`

```
1  package models;
2  import java.util.Date;
3  import com.se.Moodle.*;
4
5  classdiagram MyExamPreparation {
6
7    class MyExam {
8      Optional<Date>       secondDate;
9      List    <String>     resources;
10     Set     <Teacher>    supervisors;
11     Map     <int,String> studentIds;
12   }
13
14 }
```

CD4A Context Conditions - 1

*A context condition (**CoCo**) is a well-formedness rule that checks the static semantics of a model: rules that context-free grammars cannot capture*

- Diagram Name
  - Must match file name
  - First character upper-case
- Keywords
  - May not be used for types, e.g., “class”, “implements”
- Classes, Interfaces, Enumerations
  - Must be unique
  - First character upper-case
  - Enum constants must be unique within an enum
- Extends / Implements
  - No inheritance cycles
  - Classes may only extend classes, interfaces only interfaces
  - Only interfaces may be implemented
- Attributes
  - Start in lower-case and must be unique
  - Type must be resolvable and (optional) initialization must be type compatible
  - Overriding attribute in sub class must be of same type as the attribute in super class

CD4A Context Conditions - 2

- Associations
  - Source may not be an enumeration or external type
  - Ordered associations must have a cardinality greater than 1
  - Qualified
    - Attribute of attribute-qualifier must exist in referenced class
    - Type of type-qualifier must be resolvable
  - Names:
    - Association names and role names must start in lower-case
    - Association names, role names and implicit role names (lower-case name of target type) must not conflict with each other or attributes in the source class
  - Composition
    - Cardinality on side of the composite is [1] (default) or [0..1].
- Types
  - Generics may not be nested
  - Usage of generics must be parameterized with the correct number of type-parameters (e.g., invalid: Optional, Optional<>, Optional<A,B>, valid: Optional<A>)
  - Derived attributes may not be initialized

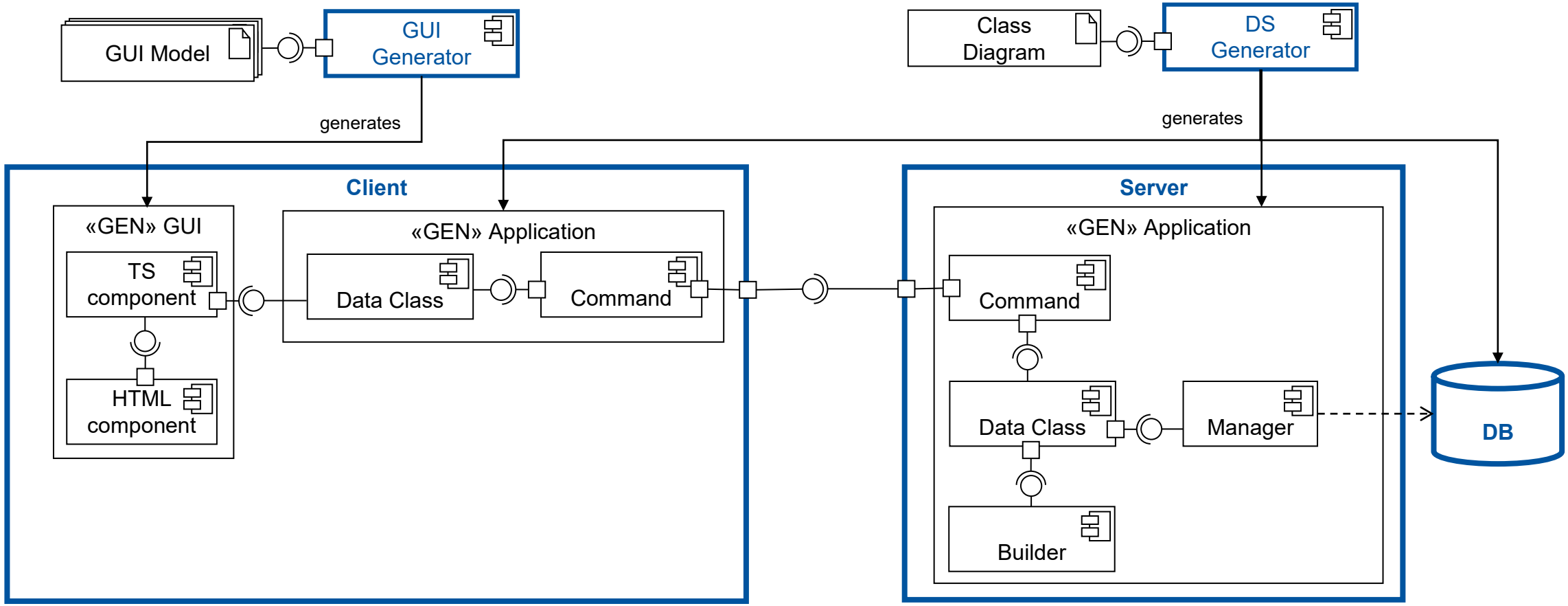
7.4 Generation process

Prof. Judith Michael | Chair of Programming and Software Engineering



Universität Regensburg

Generated Files: What MontiGem generates



Mapping a Domain Class

- Given CD `Moodle` with class `Student`:
- What should we generate for the application core?
  - *Domain class*: `Student`, represents the domain objects
    - Getter, Setter like `.getName()`, `.setEmail("mail@example.com")`
  - *Builder classes*: `StudentBuilder`, used to build (i.e. instantiate) domain objects
  - *Data Access Object (DAO)*: `StudentDAO`, used to get/set data from/to the database
  - *(De)serializer*: `StudentDeSer`, used to (de)serialize the domain objects for the transfer between backend and frontend (and other services)
  - *Observer*: `StudentObserver`, used to observe and immediately react to changes in the domain objects
  - *Commands*:
    - `Student_create`, command for creating an object
    - `Student_delete`, command for deleting an object
    - Etc.

Attributes in Domain Objects

- Generate as protected attribute
- Signature for an attribute:
  - a get-function and
  - a set-function

CD

```
class User {
  String name;
}

class Student extends User {}
```

Java
«gen»

```
1  public class Student extends User
2                implements GemIdentifiable, GemBuildable {
3
4    protected String _name;
5
6    public String getName();
7
8    public String setName(String _name){
9      this._name = _name;
10   }
11
12 }
```

Mapping a Derived Attribute

- A derived attribute generates to
  - getter signature
  - no attribute, no setter
  - no getter implementation, must be implemented in TOP class

CD

```
class Profile {
  /int numOfPosts;
}
```

The signature is implemented, but useless. Thus:

- Handcoding using the TOP mechanism is useful

Java «gen»

```
1 public abstract class Profile extends … {
2   public abstract int getNumOfPosts();
3     /* abstract method -> no body */
4
5   // But nothing to support calculation or storage of numOfPosts attribute
6   // Implement via TOP mechanism
7 }
```

Unidirectional Association, Cardinality [1]:

- Unidirectional association with cardinality [1]
  - is mapped to methods for set/get (like an attribute)
  - (additional, internal methods may be generated)
- Direction determines location of methods:
  - on the source side of each navigation
- The implementation of the methods
- Again: "null" is not used in the data structure

CD

```
association A -> B [1];
```

Java «gen»

```
1  public class A {
2
3    protected B _b;
4
5    public String getB();
6
7    public void setB(B _b){
8      this._b = _b;
9    }
10 }
```

Association, Cardinality [0..1]

- Association with cardinality [0..1]
  - is mapped to methods for set/get (like an attribute)
  - using `Optional` as type

```
association [1] Assignment -> (submitted) Submission [0..1];
```

CD

```
1  public class Assignment {
2
3    protected Optional<Submission> _submitted;
4
5    public Optional<Submission> getSubmitted();
6
7    public void setSubmitted(Optional<Submission> _submitted){
8      this. _submitted = _submitted;
9    }
10
11 }
```

Java
«gen»

Association, Cardinality [*]

- Association with cardinality [*]
  - provides signature for collection management (like the `java.util.Set` signature)

CD

```
association [1] Teacher -> Grade [*];
```

Java «gen»

```
1  public class Teacher {
2    protected Set<Grade> _grade;
3    public void                    addGrade(Grade o){…}
4    public boolean                 removeGrade(Grade o){…}
5    public boolean                 containsGrade(Grade o){…}
6    public void                    addAllGrades(Collection<Grade> o){…}
7    public boolean                 containsAllGrades(Collection<Grade> o){…}
8    public int                     sizeGrades(){…}
9    public Iterator<Grade>         iteratorGrade();
10   public void                    clearGrades(){…}
11   // …
12 }
```

Bidirectional Associations

- Redundant storage of links on both sides:
  - consistency issues
- Signature (use) remains
  - burden for consistency is on the generated implementation

CD

```
association [1] Student <-> Submission [*];
```

Java «gen»

```
1  public class Submission {
2    protected Student _student;
3    // ...
4  }
```

Java «gen»

```
1  public class Student {
2    potected Set<Submission> _submission;
3    // ...
4  }
```

Consistency in Bidirectional Associations

- Links must be consistent on both sides
  - e.g., adding a link automatically adds the link on the other object

CD

```
association [1] Student <-> Submission [*];
```

- Whenever a submission is added, an internal method is called with the NotificationScope

Java «gen»

```
1 public class Student {
2   public void addSubmission(Submission o, NotificationScope scope) {
3     if (!this._submission.contains(o.getGemId())) {
4       this._internal_addSubmission(o, scope);
5     }
6   }
7 }
```

Consistency in Bidirectional Associations

- Links must be consistent on both sides
  - e.g., adding a link automatically adds the link on the other object

CD

```
association [1] Student <-> Submission [*];
```

- In the internal method,
  1. the submission is added to the list, and
  2. depending on the scope, the current submission is retrieved via the manager, and the student is set to it
  - … and vice versa

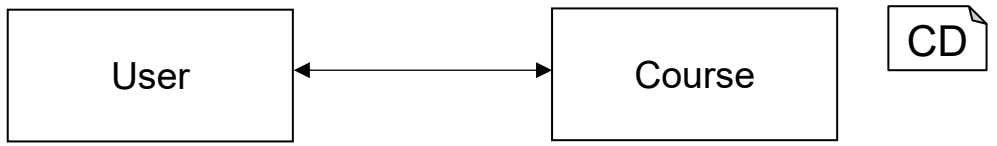
Java «gen»

```
1  public class Student {
2    public void _internal_addSubmission(Submission o, NotificationScope scope) {
3      this._submission.add(o);
4      if (scope.contains(NotificationScope.LOCAL)) {
5        MoodleManager.getSubmissionSure(o.getGemId()).setStudent((Student) this,
6          NotificationScope.MINIMAL);
7      }
8    }
9  }
```

Incompleteness (Underspecification) in CD4A

- Underspecification in CD4A models is essential for prototyping but makes code generation more complex
  - E.g., no cardinalities, no values for attributes initialization, …

→ Resolve underspecification in CD4A models using defaults to generate executable prototypes

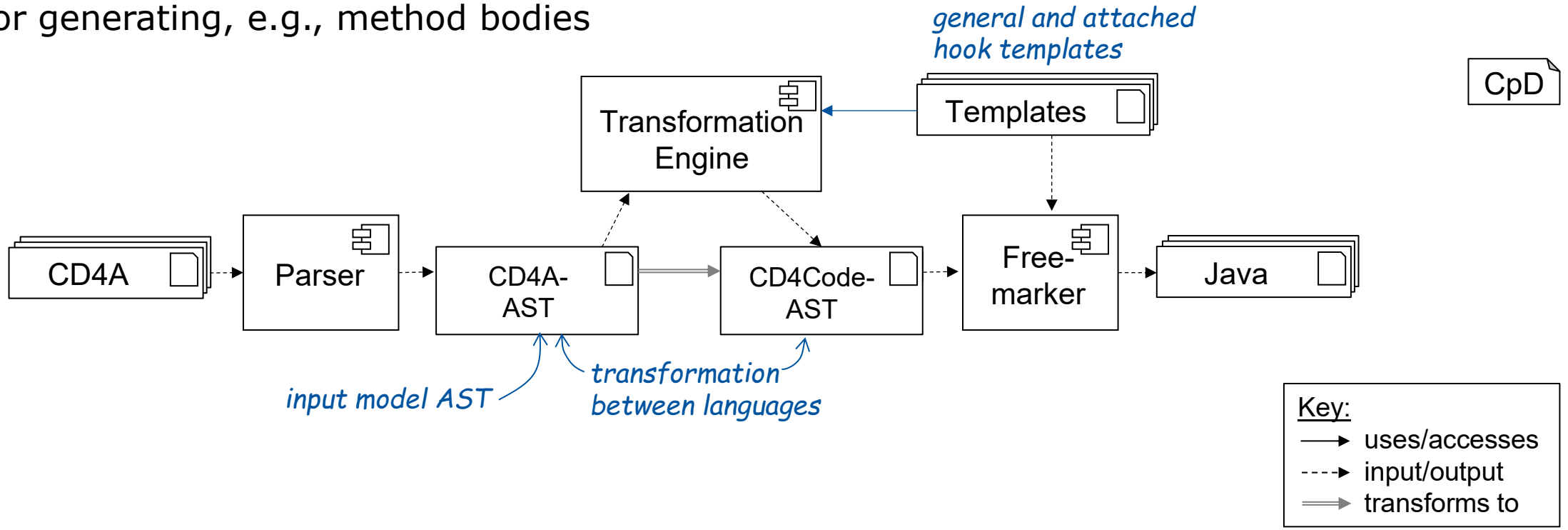


- But care: Generating defaults can add design decisions that the modeler has not necessarily intended

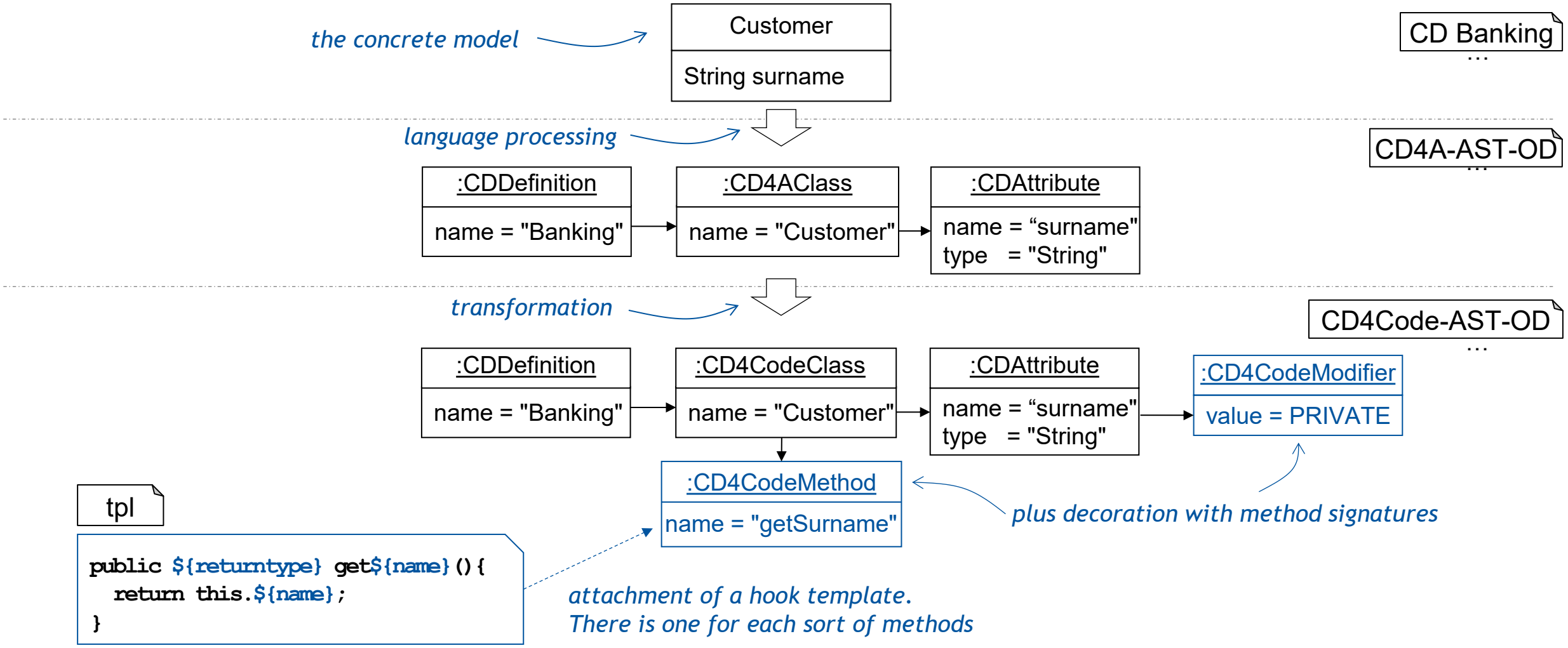
- Alternative:
  When testing, generator chooses worst behavior (runtime warnings or errors) in case of underspecification
  and behaves robustly in the product

Generation through Transformation and Templates

- The tooling parses CD4A models
- Transforms them into CD4Code models (with methods)
- Attaches specific templates to each method AST node (extensive use of hooks)
- Produces code through template engine
- Useful for generating, e.g., method bodies

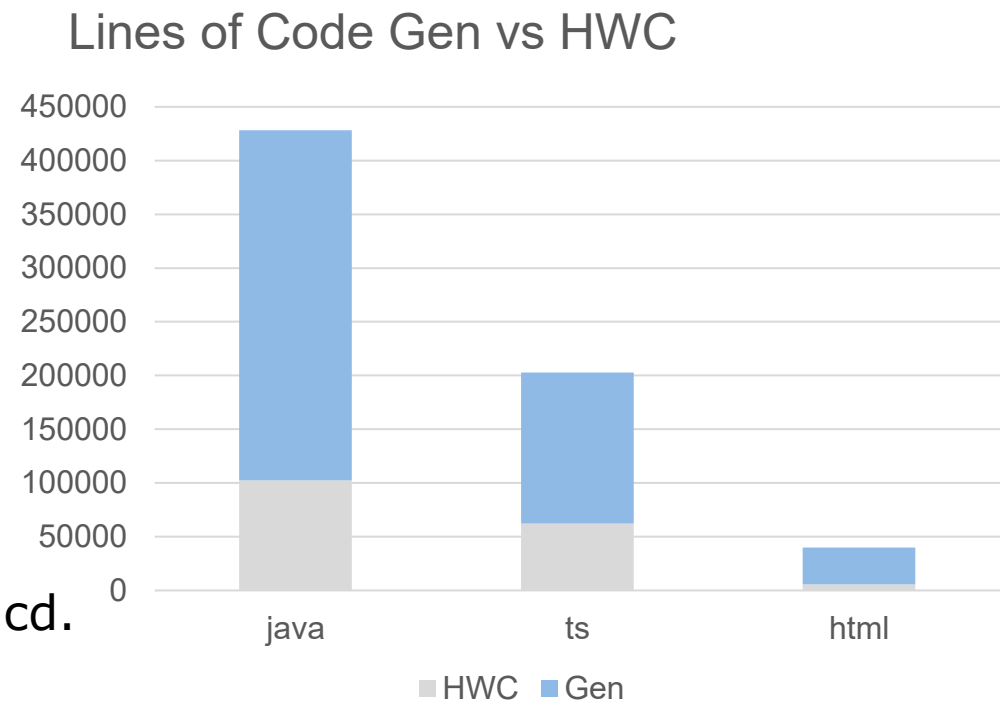


Example of Code Generation Process



Example | Generated Files : Folder Structure

- The generator stores all files into the specified (or default) output directory
  - E.g. **`build/<generated-sources>`**
- Java reflects its package structure as directories. Classes from **`MaCoCo.cd`** are generated to
  - **`backend/build/umlp2java`**
  - **`frontend/build/app/umlpJava2ts`**
  - **`frontend/build/app/gui2java2ts`**
- Source:
  - MaCoCo.cd has 72 classes and 64 assocs (Feb 22)
  - MaCoCo.cd has ~1100 LOC
- Result:
  - 4216 Java Files – 325.709 LoC
  - 3989 TS Files – 140.525 LoC
- Result: a lot of code can be generated from MaCoCo.cd.



7.5 Using the generated code

Prof. Judith Michael | Chair of Programming and Software Engineering



Universität Regensburg

Object Creation

- Each domain class has a builder
  - to create instances of classes:
  - one method for each attribute (and assoc.)
  - create the object using the build() method
- To create several related objects at once:
  - Builder objects are added like normal ones
- `build()` creates the full object structure (incl. all subobjects)

Java «gen»

```
1  public class StudentBuilder ... {
2    //...
3    public StudentBuilder name(String o);
4    public String getName();
5    public StudentBuilder email(String o);
6    public String getEmail();
7
8    // also for associations: …
9    public StudentBuilder addSubmission(Submission o);
10   public StudentBuilder addSubmission (SubmissionBuilder o);
11
12   public Optional<Student> build();
13 }
```

Managing Domain Objects

- A global MoodleManager is generated for managing and persisting all instances in the Moodle class diagram

- Create an object using a builder:

```
1 Student jim = MoodleManager.studentBuilder() // returns StudentBuilder
2   .id(123456)
3   .name("Jim Douglas")
4   .email("jim.douglas@rwth-aachen.de")
5   .build() // returns Optional<Student>
6   .get();
```

- Delete an object using the unique `gemId` of the object:

```
7 MoodleManager.deleteStudent(jim.getGemId());
```

7.4 GUI modeling

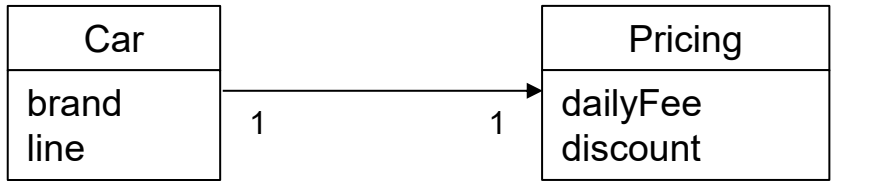
Prof. Judith Michael | Chair of Programming and Software Engineering



Universität Regensburg

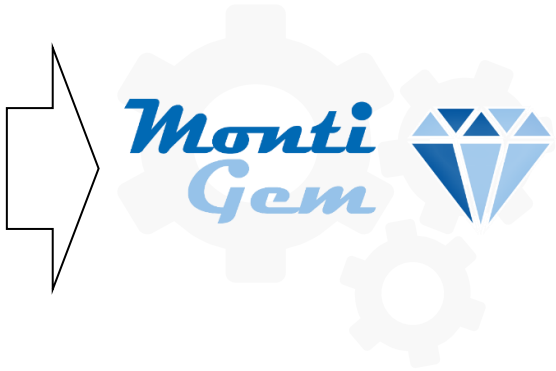
MontiGem overview

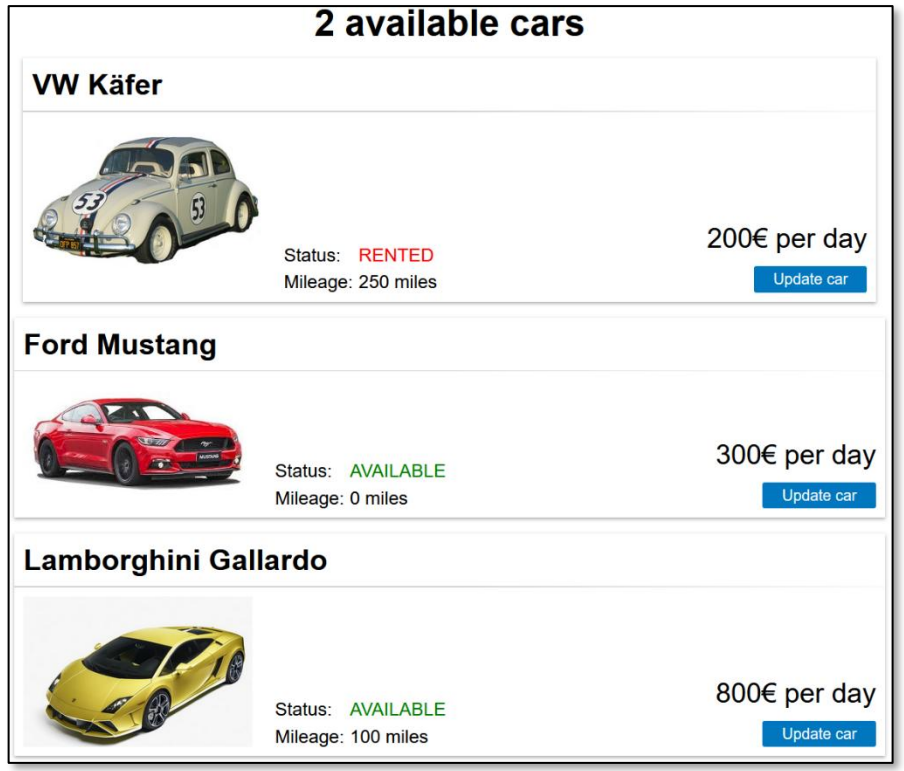
CD



GUI

```
1  package gui;
2  import CarRental.Car;
3  page CarOverview(List<Car> cars) {
4    @iterate(Car c : cars) {
5      /*…*/
6    }
7  }
```





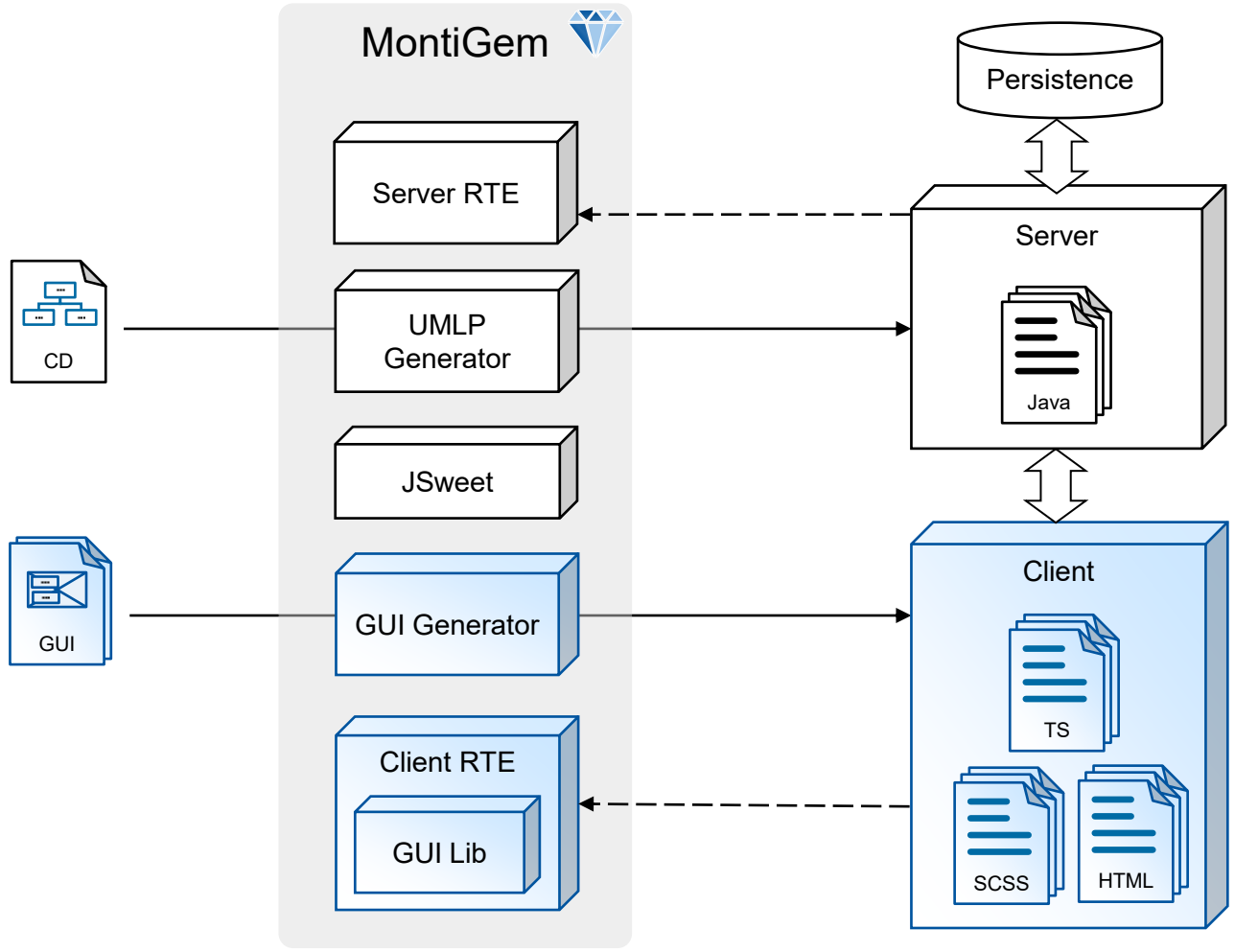
MontiGem: more detailed

CD

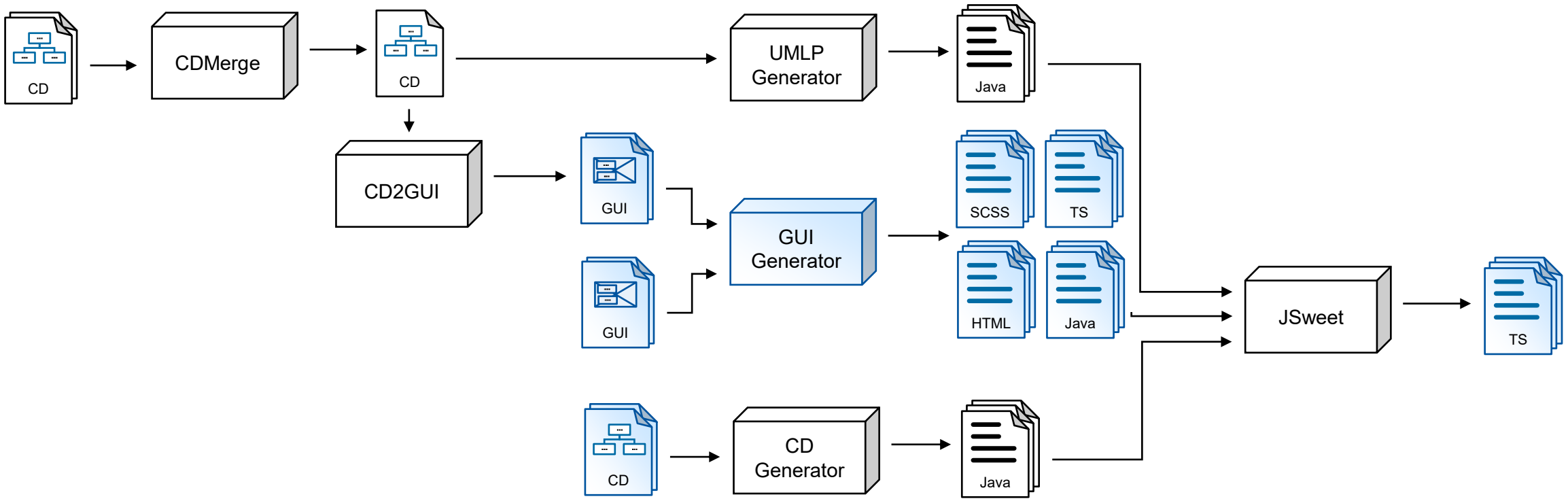
```
1  umlp CarRental {
2    class Car {
3      String brand;
4      String line;
5    }
6    /*…*/
7  }
```

GUI

```
8  package gui;
9  import CarRental.Car;
10 page CarOverview(List<Car> cars) {
11   @iterate(Car c : cars) {
12     /*…*/
13   }
14 }
```

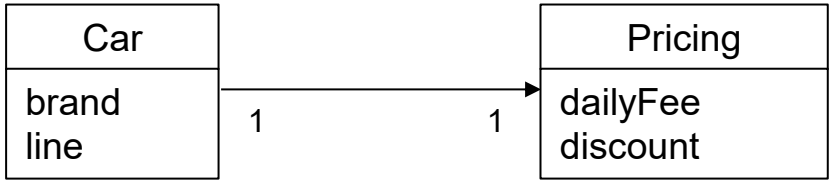


MontiGem generation process



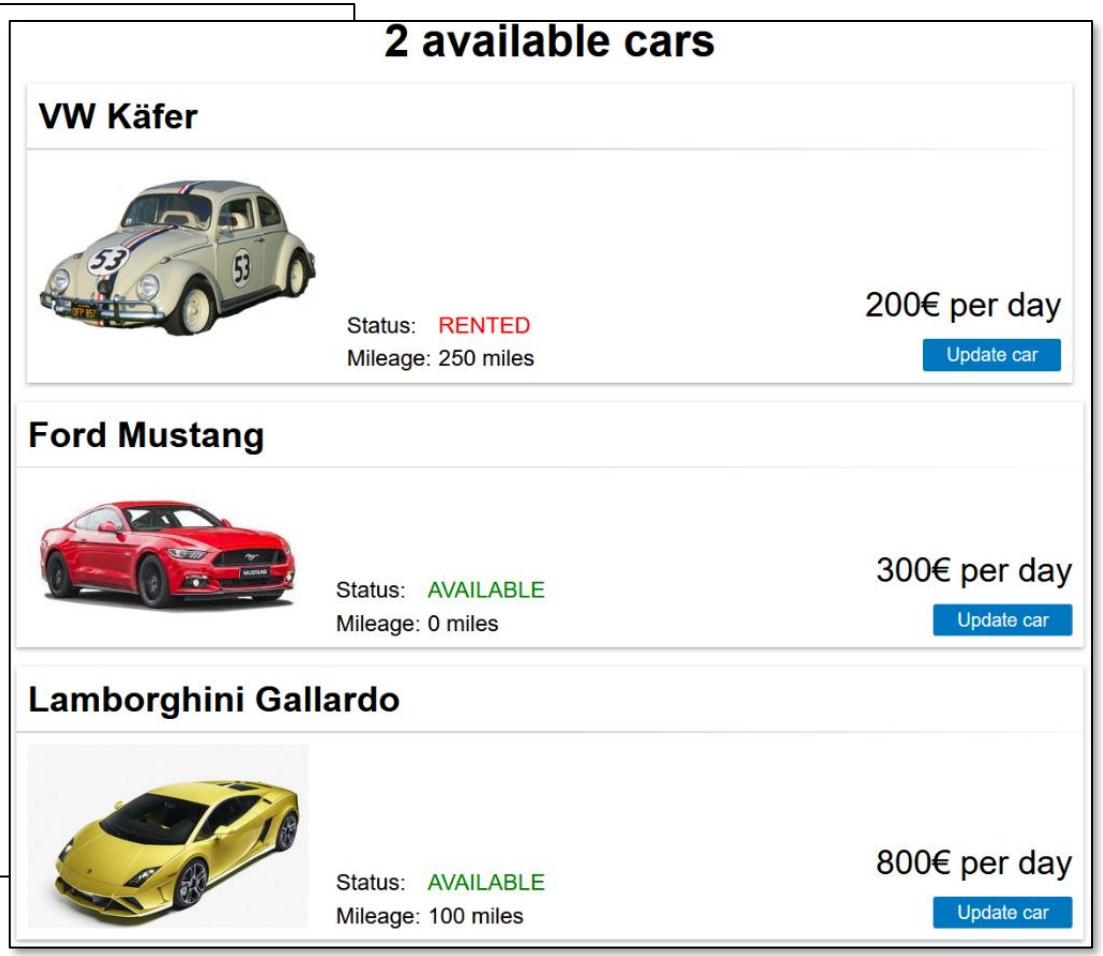
GUI DSL syntax (current state)

CD



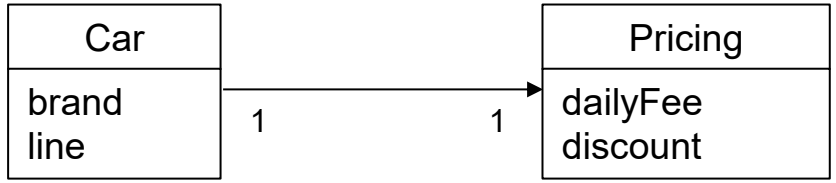

```
1  package gui;
2
3  import CarRental.Car;
4  import gui.Title;
5  import mc.fenix.basic.GemButton;
6  /*...*/
7  page CarOverview(List<Car> cars) {
8    @Title(t = getAvailableCarsTitle());
9    @iterate(Car c : cars) {
10     @ProductInfo(
11       title = c.brand + " " + c.line,
12       img = getImageUrl(c.brand),
13       /*...*/,
14       sideContent = [
15         @FfeaturedText(t = c.pricing.dailyFee + "€ per day"),
16         @GemButton(
17           label = "Update car",
18           leftClick = navigate(@CarUpdate(car = c)))
19       ]
20     );
21   }
22 }
```

GUI



GUI DSL syntax

CD



GUI

package declaration

import

page (component) declaration

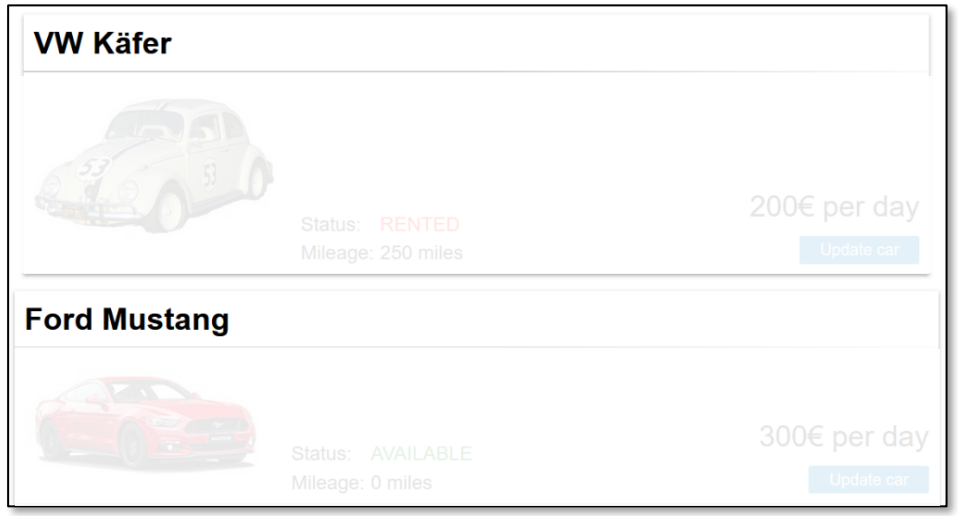
"iterate" expression

component usage

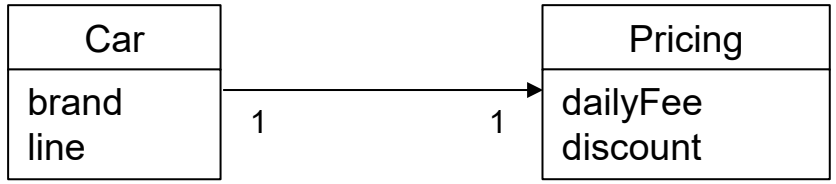
built-in method

```
1  package gui;
2
3  import CarRental.Car;
4  import gui.Title;
5  import mc.fenix.basic.GemButton;
6  /*...*/
7  page CarOverview(List<Car> cars) {
8    @Title(t = getAvailableCarsTitle());
9    @iterate(Car c : cars) {
10     @ProductInfo(
11       title = c.brand + " " + c.line,
12       img = getImageUrl(c.brand),
13       /*...*/,
14       sideContent = [
15         @FeaturedText(t = c.pricing.dailyFee + "€ per day"),
16         @GemButton(
17           label = "Update car",
18           leftClick = navigate(@CarUpdate(car = c)))
19       ]
20     );
21   }
22 }
```

Recognize data and components and draft a simple model



CD



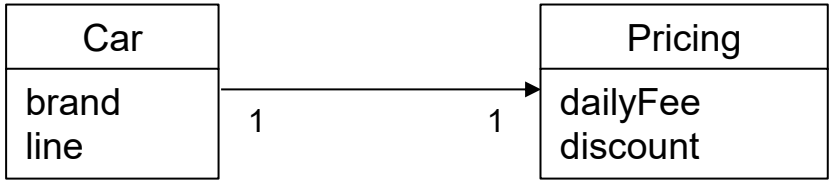
GUI

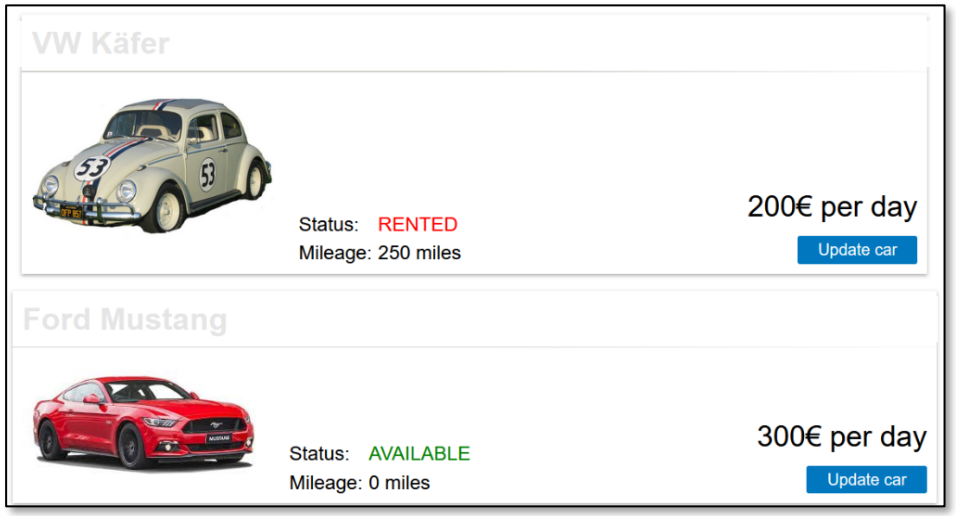
```
1  // package and imports
2  page CarOverview(List<Car> cars) {
3    @iterate(Car c : cars) {
4      @GemCard(
5        title = c.brand + " " + c.line
6      );
7    }
8  }
```

- (line 2) Shows a list of cars → **`List<Car>`**
- (line 3) **`@iterate`** through list to show each car
- (line 4) Each car has a card → **`@GemCard`**

Add GUI details, define behavior

CD





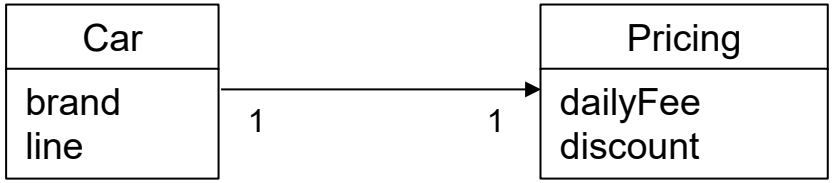
GUI

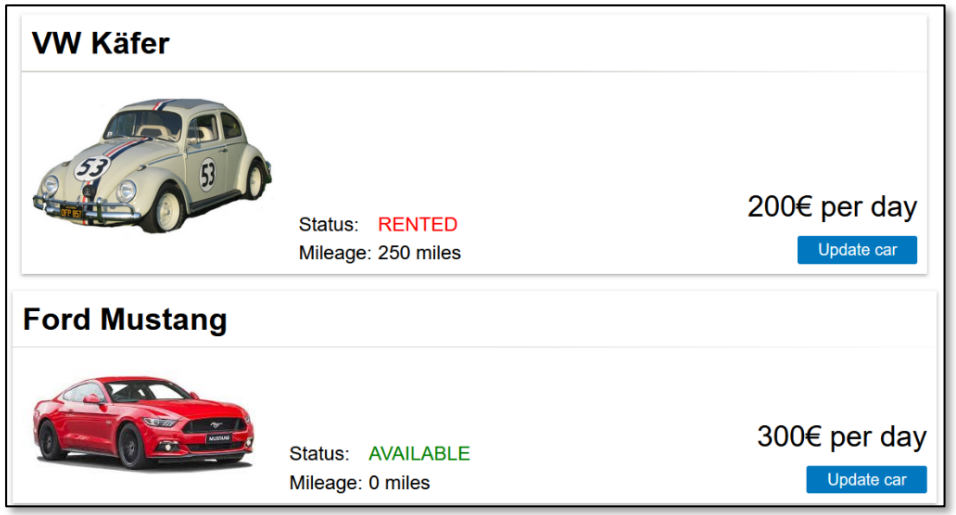
```
1  // package and imports
2  page CarOverview(List<Car> cars) {
3    @iterate(Car c : cars) {
4      @GemCard(
5        title = c.brand + " " + c.line,
6        // ... layout and styles
7        components = [
8          @GemImage(width = "200px", source = getImageUrl(c.brand)),
9          // ...
10         @GemText(value = (c.pricing.dailyFee * 3) + "€ per day"),
11         @GemButton(
12           label = "Update car",
13           leftClick = navigate(@CarUpdate(car = c)))
14       ]);
15   }
16 }
```

- (lines 8-14) add image, text, button
- (lines 10,13) simple built-in logic
- (line 8) method **`getImageUrl()`** will be generated into the code but what it does has to be defined via hand-written code using the TOP-mechanism

Fine-tune layout and styles

CD





- (lines 6, 10) Layout components
- (lines 6, 8, 10, 11) Style properties
- Extra adjustments of the style via the TOP-mechanism

GUI

```
1  // package and imports
2  page CarOverview(List<Car> cars) {
3    @iterate(Car c : cars) {
4      @GemCard(
5        title = c.brand + " " + c.line,
6        component = @GemRow(colGap = "20px", vAlign = "bottom",
7          components = [
8            @GemImage(width = "200px", source = getImageUrl(c.brand)),
9            // ...
10           @GemColumn(width = "40ch", hAlign = "right", components = [
11             @GemText(value = /*...*/ "€ per day", size = "1.5rem"),
12             @GemButton(label = "Update car" /*...*/)
13           ])
14         ]
15       )
16     );
17   }
18 }
```

GUI DSL syntax: component declarations

- A GUI component declaration
  - has a **`package`** declaration
  - uses either **`page`** or **`component`** keyword

```
1  package gui;                                          GUI
2  import CarRental.Car;
3  page CarOverview(List<Car> cars) {/*…*/}
```

```
4  package widget;                                       GUI
5
6  component ProductInfo(String title /*,…*/) {/*…*/}
```

  - may have no body (the curly braces part)

```
7  package mc.fenix.input;                               GUI
8
9  component GemTextInput(?String entry /*,…*/)
```

Can a user of my app navigate to the GUI component?

yes → **`page`**

no → **`component`**

Am I going to build the component from scratch?

yes → without body

no → with body

GUI DSL syntax: component declaration parameters

- A GUI component declaration may have input parameters
  - Generated code depends on the parameter type

GUI

```
1  package mc.fenix.input;
2
3  component GemTextInput(?String entry /*,…*/)
```

Type	Generated code	Example model excerpt
(`List` of) domain-type objects	getter/setter + loads data from the server	`page CarOverview(List<Car> cars)`
`Event<T>`	reacts to user/system events	`component GemButton(Event<Void> leftClick)`
(`List` of) `GUIViewElement`	limits parameter usage to visual representation (no getter/setter)	`component GemRow(List<GUIViewElement> components)`
`?T`	getter/setter + reacts to user/system events	`component GemTextInput(?String entry)`
Other	getter/setter	`component ProductInfo(String title)`

GUI DSL default types

- `int, Integer`
- `long, Long`
- `float, Float`
- `double, Double`
- `void, Void`
- `String`
- `java.util.List<T>`
- `Object`
  - deprecated

GUI

```
1  component GemText(
2    String value
3  )
```

- `Event<T>`
  - to be replaced via

    `(T v) -> void` and `() -> void`
  - specifies a callback for an event
  - no `List<Event<T>>`

Java

```
4  @FunctionalInterface
5  public interface Event<T>
6          extends Consumer<T> {}
```

GUI

```
7  component GemButton(
8    Event<Void> leftClick
9  )
```

- `GUIViewElement`
  - represents any
    - GUI component
    - iterate
    - guard
  - is not used in HWC

Java

```
10 public interface GUIViewElement {}
```

GUI

```
11 component GemColumn(
12   List<GUIViewElement> components,
13   GUIViewElement component
14 )
```

GUI DSL syntax: component body and other blocks

- A component body and any other block represented by `{}` may only contain objects of type **`GUIViewElement`**,
  i.e., something that represents a part of a GUI

Construct	Intent	Lines
`@iterate`	create the GUI part for each element in a list	5-13
`@guard..else`	create a GUI part if a condition is **`true`**, or another if specified	6-11
`@MyComponent`	create the **`MyComponent`**	7, 10, 12
Other expressions evaluating to **`GUIViewElement`**	put the value into GUI	15

GUI

```
1  page SimpleCarList(
2    List<Car> cars,
3    GUIViewElement footer
4  ) {
5    @iterate(Car c : cars) {
6      @guard(c.brand.equals("Ford")) {
7        @GemText(value = "Pricey!");
8      }
9      else {
10       @GemText(value = "Not pricey!");
11     }
12     @GemText(value = "Brand: " + c.brand);
13   }
14
15   footer;
16 }
```

Derive reusable components: refactor

factor out a template

GUI

```
// package and imports
page CarOverview(List<Car> cars) {
  @GemColumn(rowGap = "10px", width = "50%", hAlign = "center", components = [
    @iterate(Car c : cars) {
      @GemCard(
        title = c.brand + " " + c.line,
        component = @GemRow(colGap = "20px", vAlign = "bottom",
          components = [
            @GemImage(width = "200px", source = getImageUrl(c.brand)),
            // ...
            @GemColumn(width = "40ch", hAlign = "right", components = [
              @GemText(value = /*...*/ "€ per day", size = "1.5rem"),
              @GemButton(label = "Update car" /*...*/)
            ])
          ]
        )
```

GUI

```
// package and imports
component ProductInfo(
  String title,
  String img,
  GUIViewElement description,
  List<GUIViewElement> sideContent
) {
  @GemCard(
    title = title,
    component = @GemRow(colGap = "20px", vAlign = "bottom",
      components = [
        @GemImage(width = "200px", source = img),
        description,
        @GemColumn(width = "20ch", hAlign = "right",
            components = sideContent)
      ]
    )
  );
}
```

GUI

```
// package and imports
page CarOverview(List<Car> cars) {
  @iterate(Car c : cars) {
    @ProductInfo(
      title = c.brand + " " + c.line,
      img = getImageUrl(c.brand),
      // ...
      sideContent = [
        @GemText(value = /*...*/ "€ per day", size = "1.5rem"),
        @GemButton(label = "Update car" /*...*/)
      ]
    );
  }
}
```

Library integration

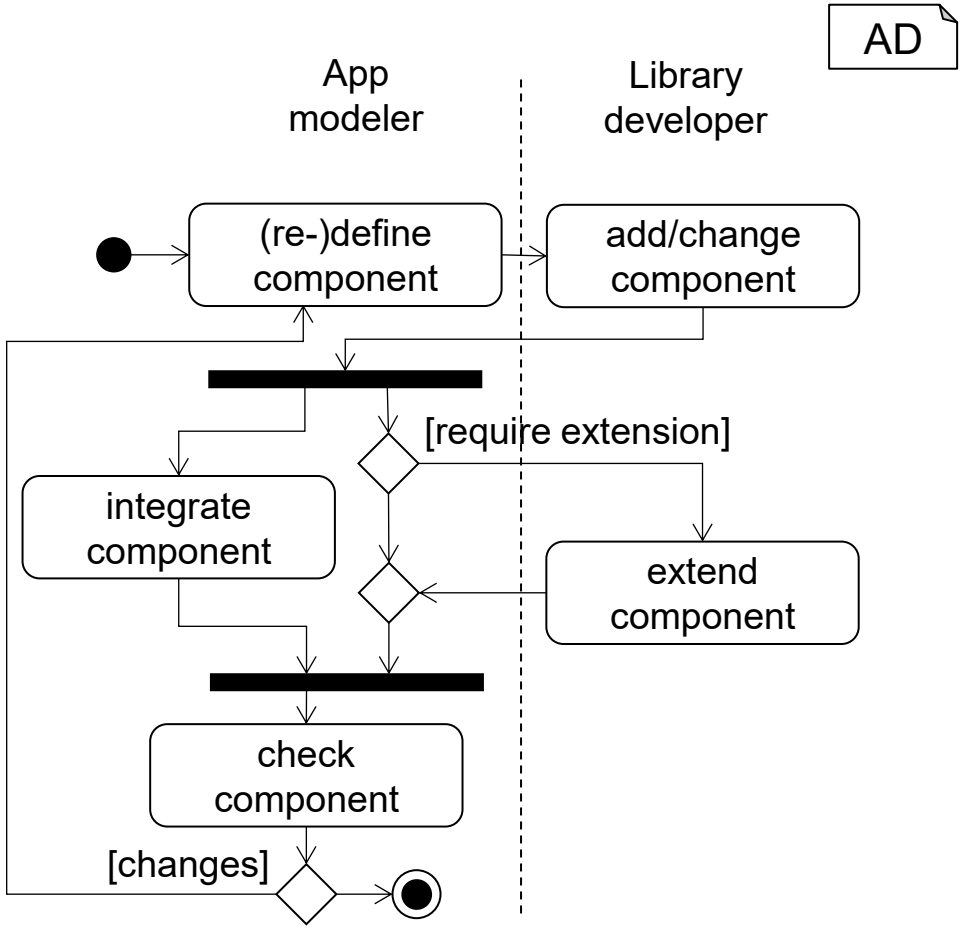
GUI

```
1  // package and imports
2  component Info(
3    String title,
4    String img,
5    GUIViewElement description,
6    List<GUIViewElement> sideContent
7  ) { /*...*/ }
```

- Domain-independent
- Reusable

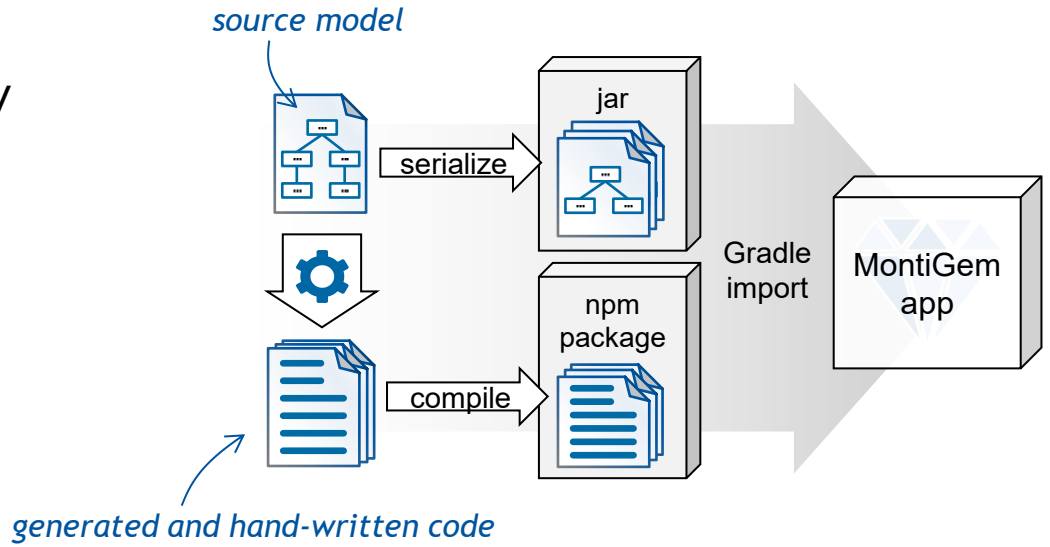
→ Library component candidate

AD



Model library deployment

- Library developer transfers the component into a library
- Library content
  - Models
  - Hand-written extensions
- Integration
  - Transfer models and HW extensions to the library
  - Push to trigger pipeline



E2E Testing

Production | *Test*

GUI

```
1  page CarUpdate(Car car) {
2    @GemTextInput(
3      labelText = "Brand:",
4      entry = car.brand
5    );
6    /*...*/
7    deleteBtn@GemButton(
8      label = "Delete",
9      leftClick = () -> deleteCar(car));
10 }
```

generate

Java «gen»

```
1  class CarUpdateTOP {/*...*/
2    public Runnable deleteBtnLeftClick() {
3      return () -> this.deleteCar();
4    }
5  }
```

Java «hwc»

```
1  class CarUpdate extends CarUpdateTOP {/*...*/
2    public void deleteCar(Car c) {
3      CarRentalManager.deleteCar(c.getGemId());
4      Router.navigate("gui", "CarOverview");
5    }
6  }
```

Java «hwc»

```
1  class CarUpdateTest extends CarrentalMockTestSetup {
2    /*...*/
3    public void CarUpdateTest(Car c) {
4      // simulate 1 client
5      super(ClientSetup.of(1));
6    }
7    // initialize dummy data
8    public void _init() throws Exception {
9      super._init();
10     CarrentalManager
11         .carBuilder()
12         .brand("Ford")
13         .line("Mustang")
14         .build()
15         .orElseThrow();
16   }
17
18   @Test
19   public void testDeleteCar() {
20     Car ford = CarrentalManager.getCar(0);
21     // activate client context
22     getClients(0).activate();
23
24     var carUpdatePage = new CarUpdate(ford);
25     carUpdatePage.init();
26
27     carUpdatePage.deleteBtnLeftClick().run();
28
29     var carOverviewPage = new CarOverview();
30     carOverviewPage.init();
31     assertTrue(carOverviewPage.getCars().isEmpty());
32   }
33 }
```

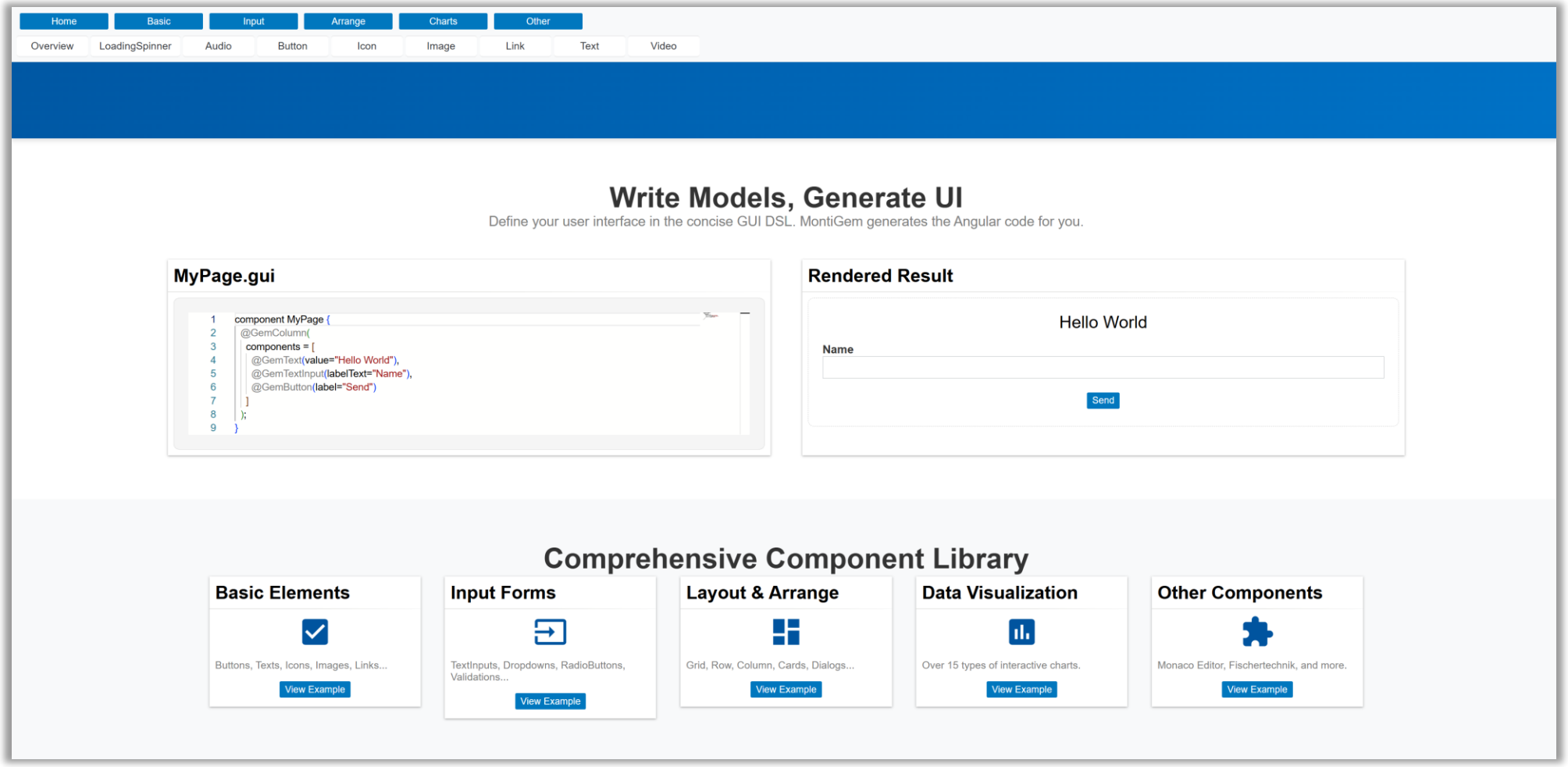
7.5 Example applications

Prof. Judith Michael | Chair of Programming and Software Engineering



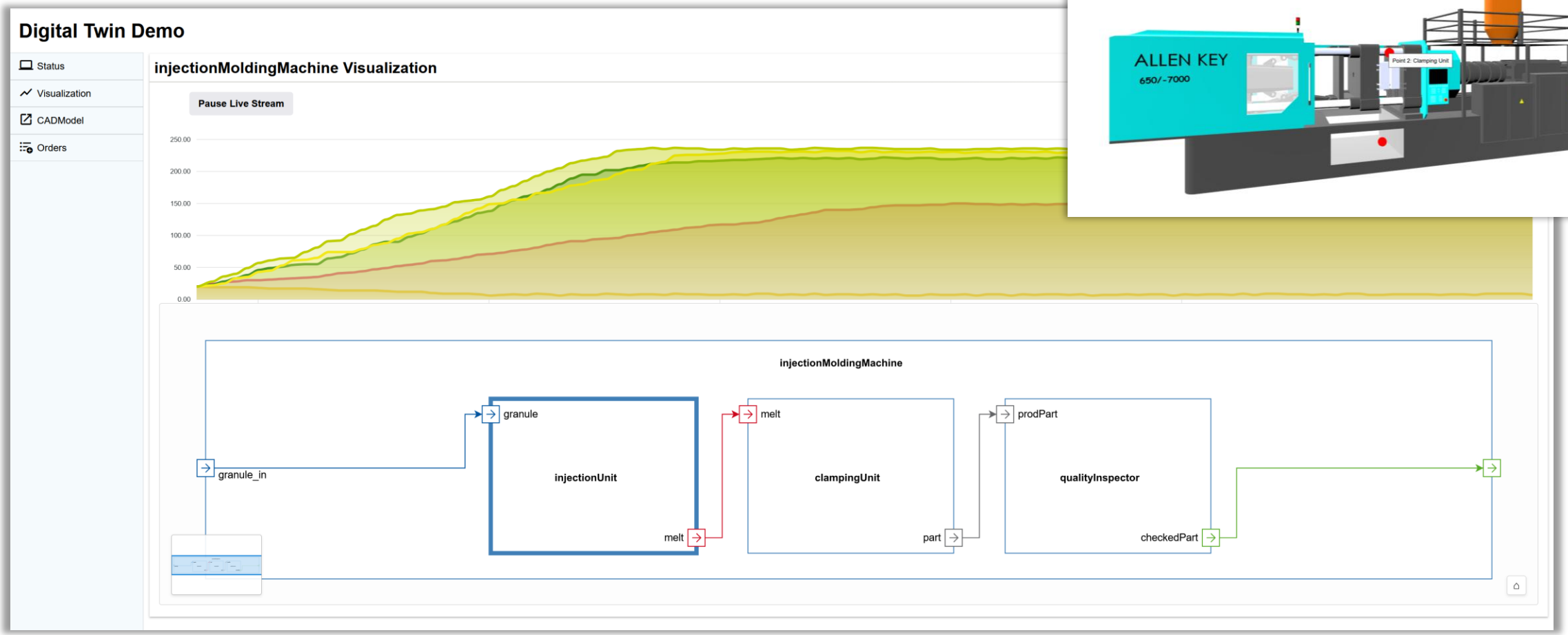
Universität Regensburg

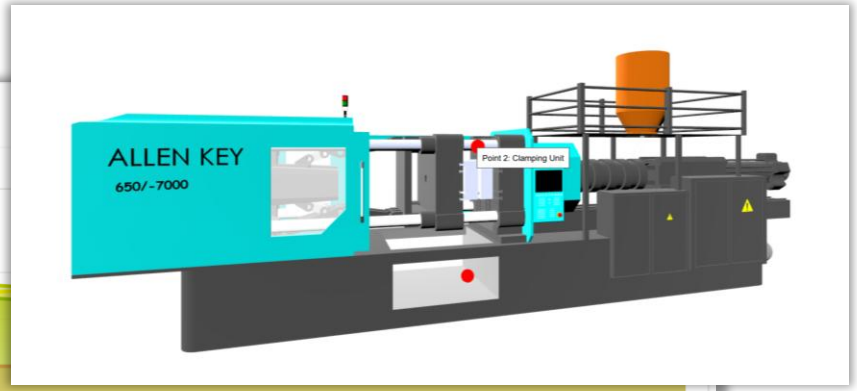
Apps Made with MontiGem 3 | Vis-Lib



https://montigem.demos.se.rwth-aachen.de

Apps Made with MontiGem 3 | Digital Twin Demo

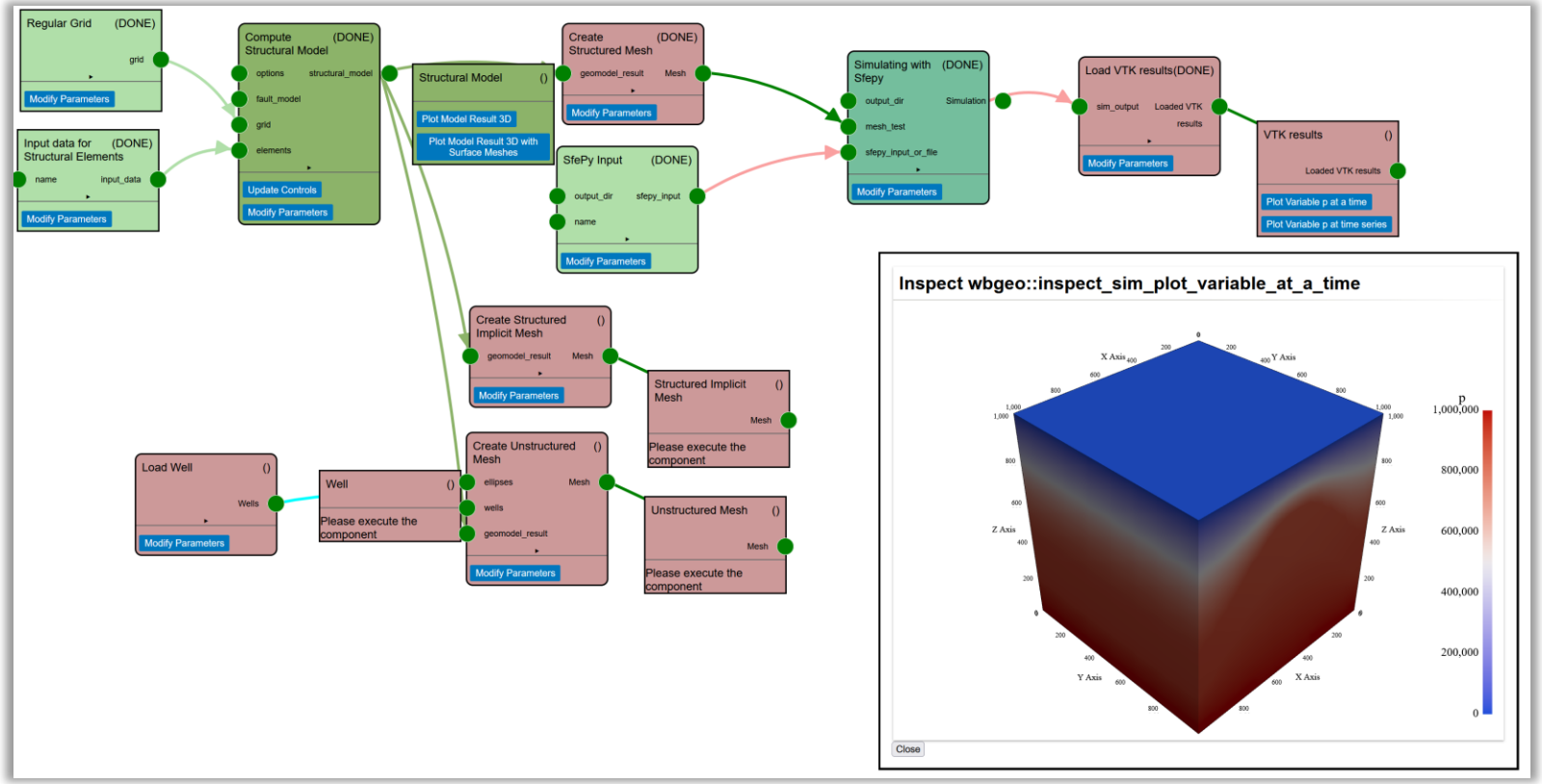




Apps Made with MontiGem 3 | WBGEO Editor

- Workflow C&C DSL
- Visual Editor for geologic modeling workflows
  - Lots of HWC and association usage
- Python Runner using umlp2py Python-Generator

Live-Demo:



Apps Made with MontiGem 3 | Artcifacts Dashboard

Language Overviews

- Current State
- Comparison
- New Analysis
- Produces
- RefersTo
- Dynamic Graph Analysis
- GrammarDashboard
- LanguageComparisons
- All Smells
- MCHexNumbers
- MCNumbers
- StringLiterals
- Cardinality
- Completeness
- JavaLight
- MCBasics
- MCCommon
- UMLModifier
- UMLStereotype
- AssignmentExpressions
- BitExpressions
- CommonExpressions
- ExpressionsBasis
- JavaClassExpressions
- LambdaExpressions

Grammar Comparisons

Grammar	Productions	Trans. Prod.	Ext. grammars	Trans. ext.	Lines of code	Last Update
Automata	3	5	1	1	34	2025.11.26
de.monticore.bpmn.Workflow	43	109	8	14	581	2026.3.19
de.monticore.CD4Analysis	0	136	7	17	48	2026.4.22
de.monticore.CD4Code	0	151	3	22	35	2026.4.22
de.monticore.fd.conformance.FDMapping	4	5	1	1	32	2025.11.26
de.monticore.FeatureConfiguration	4	85	1	8	41	2025.11.26
de.monticore.FeatureConfigurationPartial	2	89	1	9	31	2025.11.26
de.monticore.FeatureDiagram	14	71	3	7	110	2025.11.26
de.monticore.grammar.Grammar_WithConcepts	4	245	9	25	34	2026.4.27
de.monticore.GUIDSL	18	108	6	12	422	2026.2.23
de.monticore.java.JavaDSL	71	242	9	31	415	2025.1.8
de.monticore.lang.FullXML	21	55	1	5	124	2025.11.26
de.monticore.lang.JSON	12	34	1	3	106	2025.11.26
de.monticore.lang.SD4Components	7	118	6	12	96	2026.4.1
de.monticore.lang.SD4Development	10	114	5	11	127	2026.4.1
de.monticore.lang.SysMLv2	7	332	14	30	61	2026.4.3
de.monticore.lang.XMLLight	16	39	1	4	107	2025.11.26
de.monticore.mf.MontiFun	5	176	11	25	97	2025.11.18
de.monticore.ocl.OCL	10	137	6	14	167	2026.3.26
de.monticore.OD4Data	0	106	2	13	42	2026.1.7
de.monticore.OD4Development	0	106	1	14	56	2026.1.7
de.monticore.OD4Report	5	121	3	16	77	2026.1.7
de.monticore.siunit.SIUnits	10	34	1	3	337	2026.4.27
de.monticore.SymTabDefinition	2	158	3	25	83	2026.4.22
de.monticore.tagging.Tags	9	43	3	4	33	2026.4.27
de.monticore.tf.ODRules	10	217	4	24	120	2026.4.27
de.monticore.TriggeredStatecharts	1	140	7	17	61	2025.12.19
de.monticore.UMLStatecharts	1	148	12	23	56	2025.12.19
Foo	0	5	1	1	3	2025.4.10
MCEffect	4	14	1	2	17	2025.11.3
MLC	10	43	2	4	99	2025.4.10
MontiArc	1	234	15	33	29	2025.11.3
mydsl.MyDSL	6	82	7	10	32	2026.4.28
OCLWithErrorMessages	2	147	1	15	27	2026.4.28
SCMapping	7	61	2	5	21	2025.11.3
SysCL	1	164	10	21	19	2025.11.3
UMLP	0	151	1	23	17	2026.4.28

LambdaExpressions

Direct Grammar Extensions	Production Definitions	Grammars impacted
3	5	3

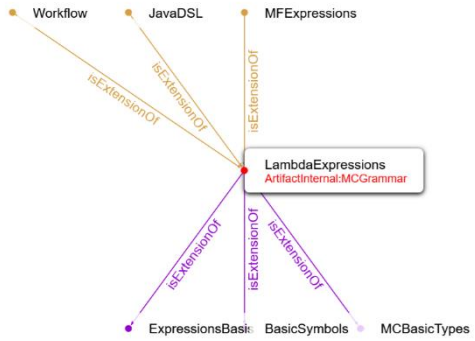
Direct Grammar Extensions

BasicSymbols
MCBasicTypes
ExpressionsBasis

Direct Nonterminals

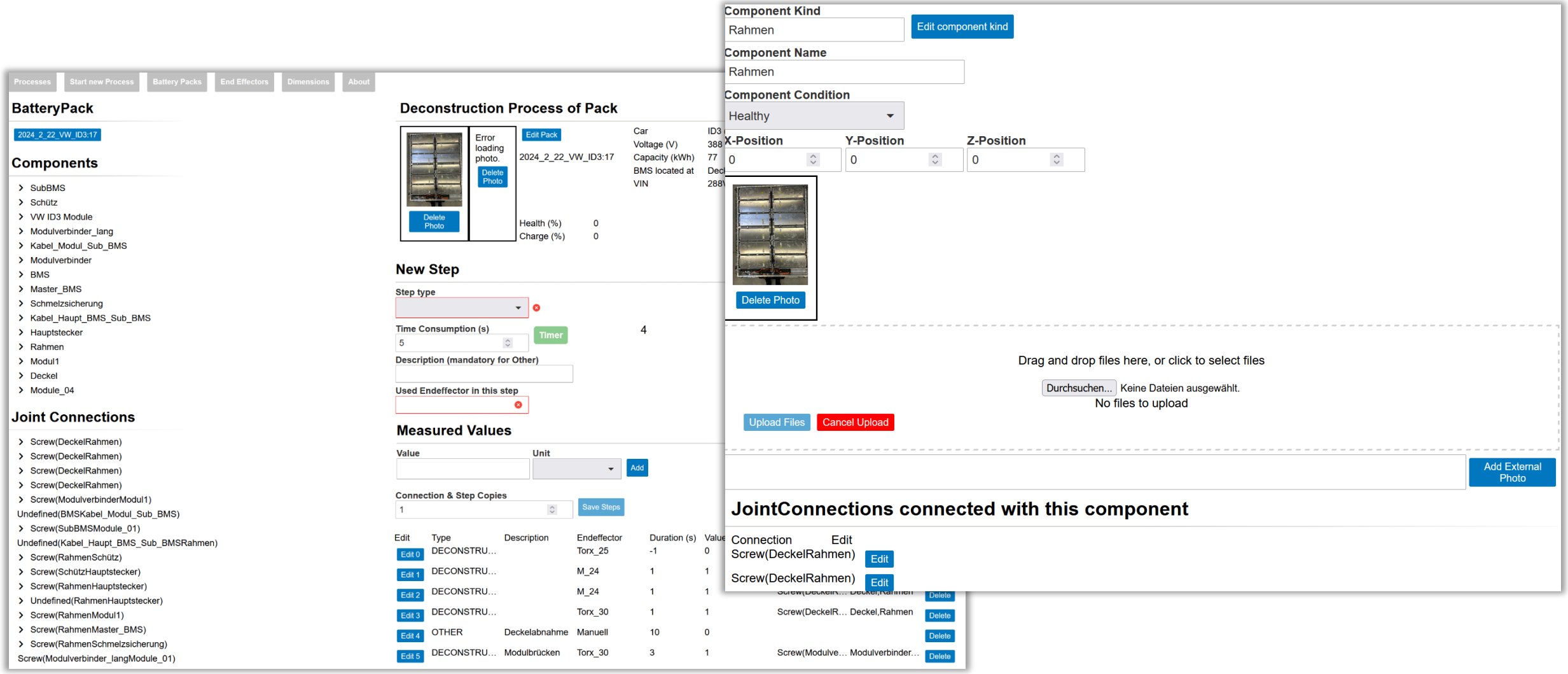
Overview Source de.monticore.expressions.LambdaExpressions.LambdaParameter
Overview Source de.monticore.expressions.LambdaExpressions.LambdaParameters
Overview Source de.monticore.expressions.LambdaExpressions.LambdaExpressionBody
Overview Source de.monticore.expressions.LambdaExpressions.LambdaExpression
Overview Source de.monticore.expressions.LambdaExpressions.LambdaBody

Transitive Impact Graph

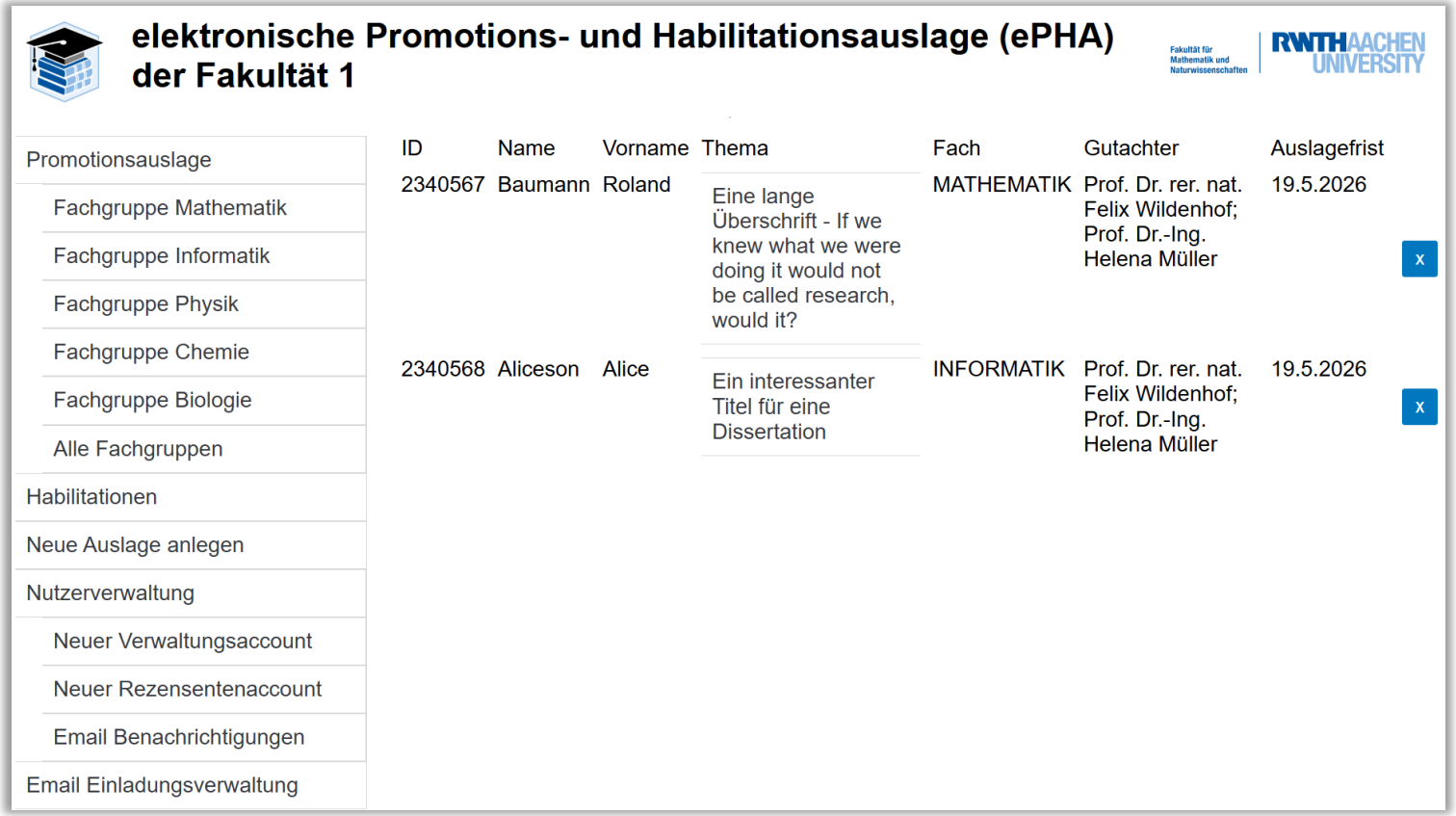


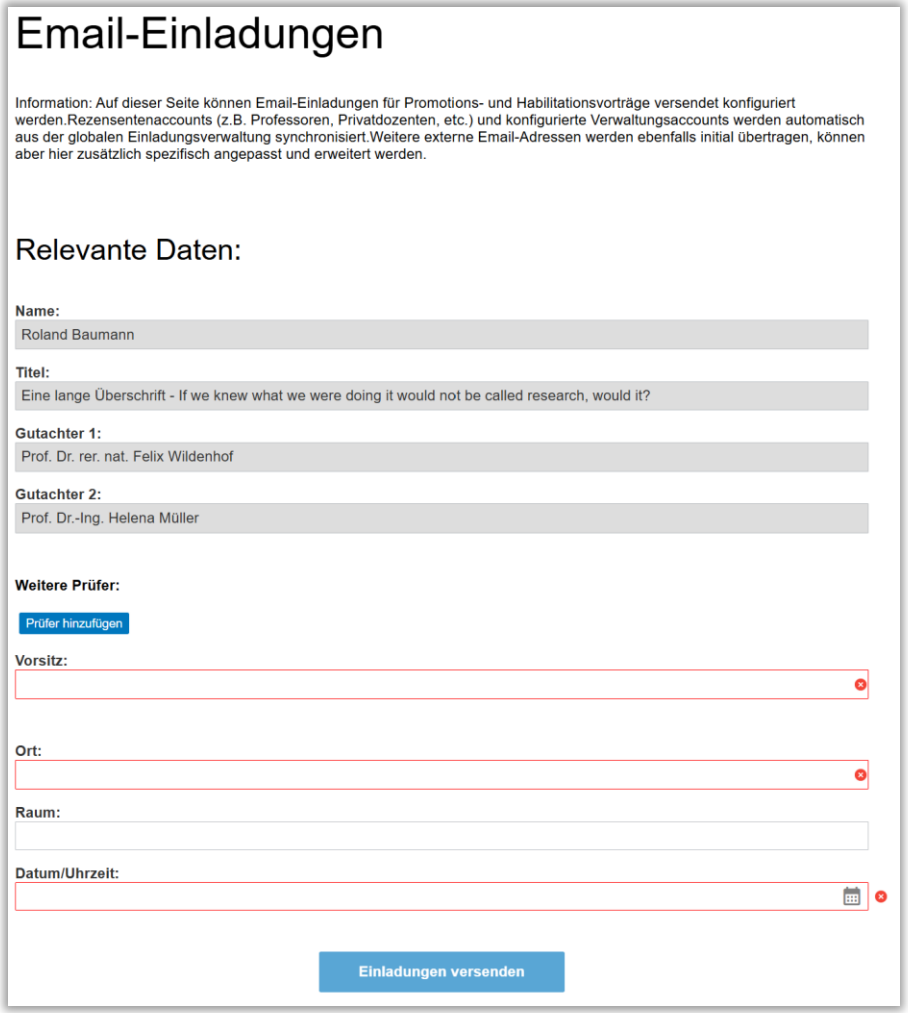
https://mc.artifacts.se.rwth-aachen.de/langs/LanguageComparisons
https://mc.artifacts.se.rwth-aachen.de/grammar/GrammarOverview/11280

Apps Made with MontiGem 3 | DemoRec



Apps Made with MontiGem 3 | ePHA





What did we cover in this chapter?

Model-Driven Engineering

- Models as driving force of the engineering process
- Code generator
- Improve quality and decrease development time

MontiGem

- Deterministic code generation
- Data-centered web-based information System
- Domain modeling with class diagrams
- GUI modeling
- Hand-written additions

Usable for different application domains

- Same RTE
- Different domain and GUI models